Real-Time Personalized Order Holding

Mohammad Reza Aminian
Booth School of Business, University of Chicago, Chicago, Illinois 60637
maminian@chicagobooth.edu

Will Ma
Graduate School of Business, Columbia University, New York 10027
wm2428@gsb.columbia.edu

Linwei Xin
Booth School of Business, University of Chicago, Chicago, Illinois 60637
Linwei.Xin@chicagobooth.edu

E-commerce retailers have a buffer to "hold" some orders before dispatching them for warehouse picking, which has benefits such as consolidating with another order placed by the same customer. Some e-retailers have begun using *personalization* to decide which orders to hold, based on a customer's predicted probability of making another order. However, due to the real-time nature in which orders arrive and the dispatching deadlines imposed, e-tailers still face a non-trivial trade-off in deciding which orders to hold.

We introduce a parsimonious online decision-making model for studying this trade-off, yielding intuitive index-based policies that decide in real-time which orders to hold. We provide constant-factor competitive ratio guarantees for these policies' performances, which do not require any forecasts regarding future incoming orders. In particular, we show that two naive index-based policies have competitive ratios of $1/2$ and $1-1/e \approx 0.63$. Our main methodological contribution is to develop a flexible framework for defining "quasi-utilities" that couples the states of an online vs. offline algorithm, which reduces the design of online index-based policies to an offline geometrical problem. The framework allows us to optimize the index function, leading to an exponential-index-based policy that has an improved competitive ratio of 0.68. We also consider various extensions and prove that most of our analyses are tight. Finally, we conduct a concise numerical study to demonstrate the strong average-case performance of the proposed policies, under non-worst-case stochastic demand sequences.

Key words: order holding, consolidation, personalization, index-based policies, online algorithms, competitive ratio, e-commerce, fulfillment, online allocation, online scheduling, reusable resources, cloud computing

History: This draft: December 24, 2024

1. Introduction

E-commerce platforms have seen tremendous growth in recent years, but with it, fulfillment costs have been growing at an even faster rate. For example, Amazon's logistics costs have skyrocketed over the past decade, with shipping and fulfillment costs growing almost 40-fold between 2009 and

2021, whereas its revenue grew only 20-fold at the same time (Statista 2022). As a result, reducing fulfillment costs is a top priority for e-retailers because it can significantly impact the bottom line.

In e-retailing, one common practice is to delay real-time order fulfillment decisions to consolidate orders (e.g., Wang et al. 2024). Instead of immediately dispatching an order to a warehouse for picking and packing, the order is held in a virtual pool for a short amount of time. If the same customer places another order shortly afterward, then a substantial amount of costs can be saved by picking and shipping the orders together. In fact, this phenomenon of *multiorder* is quite common given that many memberships (such as Amazon prime and JD plus) offer free shipping service regardless of the order size. According to Chen et al. (2024), 30% of JD.com's repeat customers place a subsequent order within 30 minutes. Meanwhile, for a leading online grocery retailer, 8% of its daily orders can be consolidated and this ratio can be as high as 17% for promotional days (Wang et al. 2024).

One straightforward idea to capture the benefits of consolidation is to offer customers a grace period, say 30 minutes, during which they can adjust their orders. While this method allows customers to consolidate multiple orders on their own, it introduces an unnecessary universal delay, during which all orders must be temporarily held even if only a handful are adjusted. Consequently, instead of holding every order, a recent industry innovation involves making personalized order-holding decisions based on the estimated chance of consolidation, i.e., the estimated probability of a customer placing multiple orders. Moreover, given a limit on the number of orders that can be held at any one time, there is a trade-off between holding a newly arrived order with a lower chance of consolidation and continuing to hold an order with a higher chance that has already been held for some time.

This paper introduces a parsimonious online decision-making model for studying this trade-off. In our core model, there is a fixed capacity C on the total number of orders that can be held. Orders $i = 1, \ldots, N$ arrive over a continuous finite time horizon. Each order i has an arrival time t_i and a rate ("height") h_i at which reward is accumulated if that order is held, capturing the chance of multiorder. (We note that our model also abstractly captures the benefits of holding orders that may be canceled (Wu 2018).) All orders can be held for a duration up to some maximum allowable delay or "lifespan," which we normalize to one, i.e., order i can be held until a time that can be as large as $t_i + 1$. If the system is at full capacity and a new order arrives, then a decision must be made on which customer's order to dispatch, possibly the order that has just arrived. The objective is to maximize the total reward accumulated from holding orders.

We elaborate on the trade-off in this problem (see Figure 1). Suppose $C = 1$ and the algorithm is currently holding order i which arrived at time t_i, and a new order j just arrived at the current time $t \in (t_i, t_i + 1)$. Keeping the current order i would accumulate reward at a rate of h_i; however,

it can only be held until time $t_i + 1$, so the remaining reward to be extracted is $h_i \cdot (t_i + 1 - t)$. Meanwhile, dispatching i and switching to the new order j would accumulate reward at a rate of h_j, with a maximum reward of $h_j \cdot 1$ that can be extracted.

If $h_j \geq h_i$, then it would be always better to hold order j, irrespective of what orders will arrive in the future. This is because order j would earn reward at a higher rate, and can also be held for a longer duration from the current time. On the other extreme, if $h_j \leq h_i(t_i + 1 - t)$, then it would be always better to hold order i, since it accumulates reward at a faster rate and also has more remaining reward to be extracted.

However, if $h_i(t_i + 1 - t) < h_j < h_i$, then we have a trade-off. On one hand, keeping order i maximizes the reward per unit time, but risks idling the holding capacity if no other order arrives before time $t_i + 1$. On the other hand, switching to order j maximizes the reward to be extracted if no future orders arrive. In this trade-off situation, we say that order i has a greater "height" (since $h_i > h_j$) but order j has a greater "remaining area" (since $h_j \cdot 1 > h_i \cdot (t_i + 1 - t)$).

We discuss the assumptions of this core model in the context of e-commerce fulfillment in Subsection 3.2 and also explore two model extensions. In the first extension (Section 6), we allow orders to have heterogeneous lifespans, capturing heterogeneity in the delivery times promised to customers. In the second extension (Section 7), we allow orders to have time-varying rates at which reward is accumulated if that order is held, capturing the fact that multiorder/cancellation probability can change over time.

1.1. Other Potential Applications

In this subsection, we highlight the generality of our model and its potential for applications beyond personalized order holding. To illustrate this, we provide several examples. We begin by examining the management of spot instances in the context of cloud computing. The recent AI wave has led to a surge in popularity for GPU rental services. Leading cloud platforms such as Amazon Web Services, Google Cloud, and Microsoft Azure offer spot instances, allowing users to utilize available excess computing capacity at a cost lower than the standard on-demand price. While spot instances are indeed cost-effective, they are also preemptible, subject to interruption whenever service providers need to reclaim capacity for higher-priority instances (e.g., on-demand and reservation instances). The demand for spot instances has surged since 2023, resulting in a fourfold increase in preemptions (Pauley 2023). Faced with the escalating frequency of interruptions, service providers are compelled to determine which spot instances to preempt upon reaching capacity limits. When such interruptions occur, users have the option to either terminate the task or restart it once capacity becomes available. If terminated, service providers like AWS and Google Cloud still charge payments for the hours used (AWS 2021, Google 2023), fitting the reward process in our model.

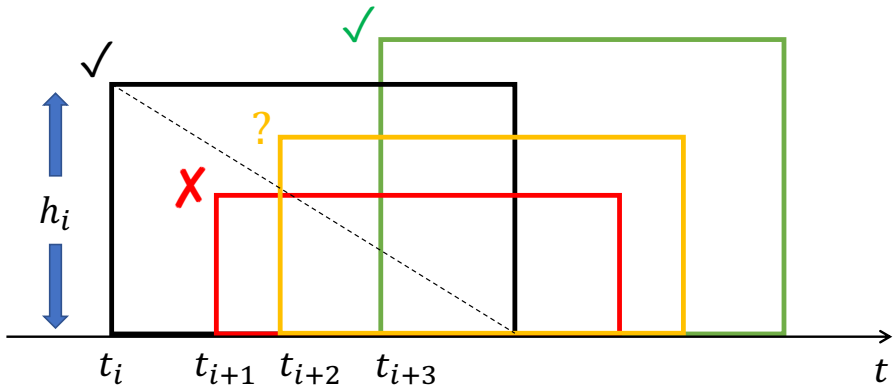


Figure 1 **Suppose $C = 1$ and the algorithm is currently holding order i with height h_i that arrived at time t_i. If another order arrives at time t_{i+1} with smaller remaining area (equivalently, the red order (t_{i+1}, h_{i+1}) is below the dashed black line), then the algorithm should continue to hold order i and reject order $i+1$. If an order arrives at time t_{i+2} and is above the dashed black line but has smaller height (i.e., the orange order), then the algorithm faces a non-trivial trade-off and the optimal action depends on future arrivals. Lastly, if an order arrives at time t_{i+3} with larger height (i.e., the green order), then the algorithm should start holding this new order $i+3$.**

Vast.ai, as another example, is an emerging cloud computing marketplace that specializes in offering GPU rental services (Vast.ai 2024). It connects a wide range of GPU providers, from individual hobbyists to large data centers, with users seeking computational resources. The platform supports both on-demand and interruptible rental options. Interruptible instances operate on a bidding system: the instance with the highest bid runs. Similarly, when interrupted, users may prefer to terminate their task and migrate the remaining portion to another machine or service provider rather than wait indefinitely while incurring storage costs. As noted by Vast.ai: "once your instance is interrupted, it could be a long wait until it resumes." This practical tendency aligns with our model's assumption that tasks may not resume after interruptions. Users are also charged for the GPU hours used, again fitting the reward process in our model.

Another potential application of our model lies in healthcare, particularly in emergency/urgent care and intensive care units (ICUs). In these critical settings, service providers face limited resources such as beds, equipment, and staff. The urgency of tasks in these units demands quick decision-making when patients with urgent needs arrive. If a patient needing immediate attention arrives and capacity is available, admission is granted. However, when capacity is fully utilized, healthcare providers encounter a delicate trade-off. This involves deciding whether to redirect the new patient to other providers or to create capacity by early discharging an existing patient whose condition has improved to hospital floors (e.g., Wagner et al. 2013, Bai et al. 2021). This complex decision-making process highlights the importance of our model in optimizing critical resource

allocation and in enabling the earlier discharge of patients in less critical conditions, all in real time.

It is noteworthy that in both cloud computing and healthcare applications, the lifespan represents the total time required by the user to be fully served, beyond which they no longer need the service or are unwilling to pay for it. This time can be estimated from the specific task, or set directly by the user or provider. For instance, Google Cloud once imposed a 24-hour limit on spot instance usage, now customizable by users (Google 2023).

Lastly, our model has potential applications in crowdsourcing and temporary labor outsourcing, areas extensively utilized by companies such as Microsoft and Nasdaq (Upwork 2023). These firms manage a wide range of projects and tasks, each with varying levels of importance and fluctuating workforce requirements. To mitigate the risks of overstaffing, they typically maintain a regular workforce smaller than what peak demand might necessitate. During times of increased demand, they turn to platforms like Upwork to hire hourly labor to complement their in-house capacity. In such scenarios, companies might outsource less critical tasks, especially those that do not require highly specialized skills, on a temporary or hourly basis. Our model offers a framework for deciding whether to outsource a new task or to conclude an ongoing one. This involves either outsourcing the remaining part of the task or halting it altogether to accommodate incoming demands.

1.2. Competitive Ratio Results

We now describe our theoretical results, starting with our core model where orders have constant reward rates and homogeneous lifespans. We consider online algorithms that do not require any forecast of the future. We analyze the competitive ratio (CR), which is the worst-case guarantee that an algorithm can provide on the total reward compared to that of an offline algorithm that knows the future in advance.

We start by analyzing π^{h}, an algorithm that always holds the orders with the largest heights, and π^{ra}, an algorithm that always holds the orders with the largest remaining areas. As discussed on the trade-off above, these two algorithms represent two extremes. We prove that π^{h} has a CR of exactly 1/2 (Proposition 1), i.e., it always accumulates total reward at least half of that of an optimal offline algorithm, and moreover there exists an instance in which the algorithm collects exactly half. Meanwhile, we prove that π^{ra} has a CR of exactly $1-1/e$ (Proposition 2).

Next, we improve on the CR using algorithms that consider both height and remaining area when facing the trade-off. We provide a novel framework for deriving index-based algorithms and analyzing their CR's (Theorem 1). In particular, we show that an exponential-index-based algorithm $\pi^{\mathsf{exp}(\alpha^*)}$ has a CR of exactly $\frac{1-e^{-\alpha^*}}{\alpha^*}$, which exceeds 0.68 (Theorem 2) under an optimized parameter α^* designed by using the Lambert W function. This beats not only $1-1/e \approx 0.63$ but

also an upper bound of 2/3 (Proposition 3) that limits any analysis based on a "Skyline" relaxation. We also provide upper bounds of 0.84 on the CR of any deterministic algorithm when $C=1$ (Proposition 4) and 0.92 on the CR of any randomized algorithm (Proposition 5).

Our first extension is to allow orders to have heterogeneous lifespans, letting L denote an upper bound on the ratio of the longest lifespan divided by the shortest lifespan. Here, we show that a competitive ratio which degrades as $O(1/\sqrt{L})$ is unavoidable (Proposition 6). We then derive a black-box reduction π^{red} that allows any policy π originating from the model with homogeneous lifespans (as described above) to be executed under heterogeneous lifespans, while losing a factor of at most $\sqrt{L}$ in the competitive ratio (Theorem 3). For example, if $\pi=\pi^{\mathsf{exp}(\alpha^*)}$, then the competitive ratio would be at least $0.68/\sqrt{L}$. We note that this holds even if $C=1$, and that the $O(1/\sqrt{L})$ upper bound also requires $C=1$. Motivated by this, we derive a second reduction algorithm π^{div} specifically for large-capacity regime, which leads to a slower competitive ratio degradation rate of $\Omega(1/\log(L))$ under the assumption that $C=\Omega(\log(L))$ (Theorem 4).

Our second extension is to allow orders to have time-varying reward rates, under a *homogeneity* assumption that the rewards of all orders scale at the same rate over time. To elaborate, we assume for an order i arriving at time t_i that its reward rate at time t is $h_i\cdot\mathfrak{r}(t-t_i)$, for all $t\in[t_i,t_i+1)$. We identify a condition on the scaling function $\mathfrak{r}$ that allows our exponential-index-based algorithm and analysis to automatically extend, with exponential functions $\{e^{-\beta t}\}_{\beta\in\mathbb{R}}$ being a class of functions that satisfy this condition. The competitive ratio depends on a parameter R, the integration of the scaling function $\mathfrak{r}(t)$ over $[0,1]$ (to be formally defined in (12)). When $R<1$, the competitive ratio *improves* beyond 0.68. In fact, $\mathcal{CR}$ goes all the way to 1 as R shrinks to 0. In the special case of exponential function $\mathfrak{r}(t)=e^{-\beta t}$, $\mathcal{CR}$ improves from 0.68 to 1 as the decay rate β increases from zero to infinity. The intuition for this result is that the online algorithm's trade-off becomes easier if the reward from holding orders decays so quickly, as the desiderata of "height" and "remaining area" coalesce upon one another.

We provide an abridged summary of the competitive ratios discussed above in Table 1.

Table 1 An abridged presentation of our competitive ratio results in the core model and extensions. Here, $\mathcal{CR}(\pi)$ represents the competitive ratio of a policy π. The results are exact unless denoted by $\leq$ or $\geq$.

Setting	Competitive Ratio	Result	Reference
Core Model	$\mathcal{CR}(\pi^{\mathsf{h}})$	0.50	Proposition 1
	$\mathcal{CR}(\pi^{\mathsf{ra}})$	0.63	Proposition 2
	$\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)})$	0.68	Theorem 2
	$\sup_\pi \mathcal{CR}(\pi)$ (Upper Bound, $C=1$)	≤ 0.84	Proposition 4
Heterogeneous Lifespans	$\mathcal{CR}\left(\pi^{\mathsf{red}}(\pi^{\mathsf{exp}(\alpha^*)})\right)$	$\geq 0.68/\sqrt{L}$	Theorem 3
	$\sup_\pi \mathcal{CR}(\pi)$ (Upper Bound, $C=1$)	$\leq 1/\sqrt{L}$	Proposition 6
	$\mathcal{CR}\left(\pi^{\mathsf{div}}(\pi^{\mathsf{exp}(\alpha^*)})\right)$ (when $C=\Omega(\log(L))$)	$\Omega(1/\log(L))$	Theorem 4
Time-Varying Rewards	$\mathcal{CR}\left(\pi^{\mathsf{exp}(\alpha^*(R))}\right)$	$\frac{e^{\alpha^*(R)}}{e^{\alpha^*(R)}+1}$ ♣	Theorem 5

♣ We refer to Figure 3 on how $\alpha^*(R)$ and $\mathcal{CR}\left(\pi^{\mathsf{exp}(\alpha^*(R))}\right)$ change as a function of R, where R represents the integration of the scaling function $\mathfrak{r}$ over $[0,1]$ as defined in (12).

1.3. Sketch of Analysis Technique

At its heart, our analysis is based on the following argument. Suppose that the online algorithm always kept the (up to) C orders with the largest reward rates, and had the following privilege—if an order hits its deadline while being held by the algorithm, then its lifespan would be "extended" by one unit of time and would still be held using imaginary capacity. We show such an online algorithm upper-bounds the performance of any offline algorithm. Of course, this privilege is imaginary, and could overestimate the actual reward of the online algorithm by a factor of 2. Nonetheless, this argument immediately implies that the policy which keeps the C largest reward rates has a competitive ratio at least 1/2.

Next, we observe that the above argument is preserved if rewards are replaced by "quasi-utilities"—an online algorithm that keeps the C largest quasi-utility rates and has the aforementioned privilege upper-bounds the quasi-utility that can be earned by any offline algorithm. If we make the definition of quasi-utility smaller than the actual reward, then the overestimation factor described above could be smaller, and its reciprocal which we denote by γ_a (one of the two key factors in the competitive ratio guarantee) could be bigger. However, the optimal offline's quasi-utility would then be smaller than the optimal offline's actual utility, requiring us to multiply the guarantee by γ_o (the other key factor in the competitive ratio guarantee).

Importantly, this argument works for *any* definition of quasi-utility, and the factors γ_a and γ_o are easy to calculate from the quasi-utility function because they only compare online to online and offline to offline, respectively. In this way, our framework has achieved "disentanglement" of online and offline decisions. We highlight that our framework transforms the online decision-making problem to an offline geometrical problem, in which one shall design quasi-utility functions that

have a large final competitive ratio $\gamma_a\gamma_o$, leading to an exponential quasi-utility function with an optimized decay rate. Our resulting algorithm will then be the policy that keeps the C orders with the highest indices according to these exponential functions.

Our flexible analysis framework allows us to define similar exponential quasi-utility functions in the extension where actual reward rates are time-varying, as we demonstrate in Section 7. We compare our analysis framework to existing literature in Subsection 2.4.

2. Literature Review

2.1. E-commerce Fulfillment

Our work primarily contributes to the literature on online order fulfillment. Xu et al. (2009) are the first to define and study the online fulfillment optimization problem. There has been growing interest in this topic in recent years, given that online shopping is becoming more popular. Those include Acimovic and Graves (2015), Jasin and Sinha (2015), Andrews et al. (2019), Zhao et al. (2020), Amil et al. (2022), Gao et al. (2022), DeValve et al. (2023), Ma (2023), and Xie et al. (2024). We refer to a tutorial on the fulfillment optimization problem by Acimovic and Farias (2019).

Wu (2018) documents the practice of personalized order holding at JD.com, a leading Chinese e-retailer, and explains that the length of delay of an order should depend on the probability of order cancellation/consolidation. Wei et al. (2021) investigate the optimal consolidation policy in the setting of two warehouses, and their model captures the trade-off between economies of scale from consolidating orders and the expedited shipping costs due to shorter deadlines. Feng et al. (2024) study how to consolidate truckloads in the middle-mile, which involves moving packages from regional warehouses to last-mile delivery stations. They demonstrate that adding a little flexibility to the routing network can significantly reduce transportation costs, using data from Alibaba Cainiao. Wagner et al. (2023) find that delivering all products in an order together (even if later) can reduce the probability of returns. Amorim et al. (2022) look into the context of online grocery retailing, where home deliveries often require the presence of the customer. They find that customers not only prefer time slots that shorten the time between order and delivery, but also favor time slots with a narrow window for the offered delivery. Chen et al. (2024) model the order-holding problem (for each individual customer) as a Markov decision process and show that the optimal order-holding decisions follow a threshold-type policy. The most relevant paper to ours is Wang et al. (2024), who formulate the order consolidation problem as a Markov decision process where all new orders are held in a temporary order pool and the release timing is determined by solving the dynamic program. However, their approach suffers from the curse of dimensionality. To address this, they propose a heuristic solution by using an LP-based approximation. They also evaluate the benefits of personalized order holding by using real data. Comparing with their model, ours is

parsimonious while incorporating the fundamental elements from theirs, e.g., identical maximum holding duration for all orders, a capacity on the maximum number of orders the system can hold, and that order consolidation probability does not change over time. In this way, we are able to highlight the trade-off between holding a newly-arrived order with a lower chance of consolidation vs. holding an order with a higher chance that has already stayed a while in the pool. As such, our online algorithms are intuitive and highly interpretable. Most importantly, our algorithms have worst-case performance guarantees, whereas their heuristic does not.

We note that the practice of holding orders enjoys several benefits: providing a backlog to enable judicious job sequencing, increasing market thickness, allowing order cancellations and consolidations, and providing additional information to the online algorithm (see Xie et al. (2024) for a classification). Our paper specifically focuses on the third benefit, modeling it through the concept of a holding reward, in contrast to Xie et al. (2024), who emphasize informational benefits.

2.2. Online Scheduling

Our problem is closely related to the classical line of work on "competitive analysis" for online scheduling (see Kolen et al. 2007). Specifically, our model can be described as having sharp deadlines (jobs must be accepted or rejected immediately upon arrival) with no discounting in the model of Dasgupta et al. (2009), which allows for job cancellations with partial rewards. Note that Dasgupta et al. (2009) already contrasts with many other papers, such as Woeginger (1994), Canetti and Irani (1995), Epstein et al. (2016), which do not permit job cancellations with partial rewards. Our combination of features, namely the immediate start of jobs, fix deadlines, continuous service periods, the possibility of early termination, and the option to receive partial rewards, has been largely overlooked in the classical literature, even though it is the norm in many modern applications including e-commerce order holding and GPU cloud computing. We also note that the trade-off we identify (see Figure 1) differs from the focus of these classical papers.

We next provide a concrete comparison to the results in Dasgupta et al. (2009). Assuming heterogeneous lifespans while maintaining a constant reward rate over time, Dasgupta et al. (2009) derive competitive ratio bounds with a logarithmic dependence on the term $\frac{\max_i h_i}{\min_i h_i}$, the ratio of the largest to smallest reward rates. Their algorithm essentially partitions the heights uniformly on a logarithmic scale, and then selects one of these height intervals (with a height ratio of 2) uniformly at random. Within the chosen interval, it disregards the specific heights and prioritizes the order with the latest deadline among the current orders in that interval. Intuitively, by being greedy with respect to (w.r.t.) the lifespans, their algorithm's performance guarantee does not depend on L, the ratio of the longest to the shortest lifespan, but depends on $\frac{\max_i h_i}{\min_i h_i}$, the ratio of the largest to the smallest height. By contrast, our algorithm's guarantee depends on L but not on $\frac{\max_i h_i}{\min_i h_i}$.

In particular, when $L=1$, we derive guarantees that exceed $1-1/e$, a naturally-occurring barrier in the analysis of similar problems. We note that in our applications, we generally expect $\frac{\max_i h_i}{\min_i h_i}$ to be large (due to heterogeneity across users), while L is likely to be small (due to similarity in delivery options), making our form of guarantee more appropriate for our problem. Our algorithm also has the benefit of being deterministic rather than randomized, and demonstrates average-case performance that significantly exceeds its worst-case guarantee.

Table 2 Implications of our core model (Subsection 3.1) in setting of Dasgupta et al. (2009) when resources do not expire (i.e., $\beta=1$ in Dasgupta et al. (2009)).

	Dasgupta et al. (2009)	Our Paper
Setting	Randomized Algorithms	Deterministic Algorithms
Lower Bound on CR	$\frac{1}{2\left\lceil \log \frac{\max_i h_i}{\min_i h_i} \right\rceil}$	$\geq 0.68/\sqrt{L}$ (for $C \geq 1$) $\Omega\left(1/\log L\right)$ (for $C=\Omega\left(\log L\right)$)
Upper Bound on CR ($C=1$)	$O\left(\sqrt{\frac{\log\log L}{\log L}}\right)$	$\leq \min\left\{0.84,\ 1/\sqrt{L}\right\}$ ≤ 0.92 (for randomized alg.)

2.3. Online Resource Allocation with Reusability

Our paper also contributes to the recently growing literature on resource allocation with reusable resources in various contexts, from ride-sharing (Dickerson et al. 2021), online matching (Goyal et al. 2020), pricing (Besbes et al. 2019, Lei and Jasin 2020), network revenue management (Baek and Ma 2022), assortment optimization (Rusmevichientong et al. 2020, Feng et al. 2022, Gong et al. 2022), to crop gleaning (Ata et al. 2019) and volunteer crowdsourcing (Manshadi and Rodilitz 2020). Accept/reject decisions for reusable resources under adversarial arrivals are also studied in Huo and Cheung (2022), both with and without advance booking, and Ekbatani et al. (2024) considers the assignment problem with multiple servers. Abels et al. (2023) allow for selecting the duration of resource allocation at the time of the job's arrival. Golrezaei and Yao (2021) consider a setting where some requests can wait for a single unit of time if their service is guaranteed. In all of the extant literature, once a reusable resource is assigned to a job, it cannot be reassigned to a new job until the current job is completed. By contrast, in our model/application, the server can dispatch current orders at any time to make room for more rewarding orders while still collecting partial rewards for the duration of service provided.

2.4. Comparison of Techniques

We compare our analytical framework, outlined in Subsection 1.3, to existing techniques in the literature. There are papers (e.g., Chrobak et al. 2003, 2007, Gong et al. 2022, Simchi-Levi et al. 2023) that establish competitive ratios by directly comparing the optimal offline algorithm's reward

against that of the algorithm. This charging argument creates a many-to-one mapping of partitioned rewards, across time, demand requests, or resources. For instance, by ensuring that each reward unit of OPT corresponds to at most $\frac{1}{\mathcal{CR}}$ copies of the online algorithm's rewards for some $\mathcal{CR} \leq 1$, a competitive ratio of $\mathcal{CR}$ is established.

Another common technique for establishing competitive ratios in online algorithms is primal-dual approach. Here, the goal is to compare the algorithm's performance to the dual of a natural LP upper bound for the optimal offline value. The algorithm's rewards are partitioned among the dual variables so that the algorithm achieves the same total value by construction, after which one argues that these dual variables approximately satisfy feasibility (e.g., Devanur et al. 2013). An alternative approach is to guarantee dual feasibility while arguing that the algorithm's gain is at least $\mathcal{CR}$ times the dual gain at every time step (e.g., Buchbinder et al. 2013). For both alternatives of the primal-dual framework, we note that exponential index functions naturally arise as a by-product (e.g., Mehta et al. 2007, Golrezaei et al. 2014).

Nonetheless, our proof technique is tailored to our specific problem and does not directly align with any known versions of the charging or primal-dual arguments. To recap, we introduce a quasi-utility metric and an "extended version" of any policy that is specific to our problem, after which there are three layers to our argument. The first layer shows that the proposed policy's actual utility approximates the quasi-utility of its extended policy. The second layer establishes that the extended policy's quasi-utility exceeds that of the optimal offline. The third layer demonstrates that the optimal offline's quasi-utility approximates its actual utility. In addition, we provide a geometrical interpretation, reducing the complex online decision-making problem into an "offline" geometry problem, involving a parallel structure on a plane (see Subsection 4.2 for further details).

3. Model and Simple Index-based Policies

In this section, we formally introduce our model and notation, and start by analyzing some simple index-based policies. All proofs in the paper will be deferred to the appendix.

Let us define $[N] \triangleq \{1, 2, ..., N\}$ and the instance of our problem, denoted by $\mathcal{I} = \big(N, \{(t_i, h_i, \ell_i)\}_{i\in[N]}\big)$, as a tuple containing N number of arriving orders to the system in total and a set containing key information about each of these N orders, where t_i is the arrival time of order i to the system, h_i is the height (reward rate) of order i, and ℓ_i is the lifespan of the order i. We allow multiple arrivals at the same time. Lifespan of an order $i \in [N]$, denoted ℓ_i, is defined as maximum length of time that the system can keep holding (or delaying its dispatch) since its arrival time t_i. We refer to the time $t_i + \ell_i$ as the deadline (or end of lifespan) of order i, and order i is said to be "active" during interval $[t_i, t_i + \ell_i)$. Height h_i is the reward of holding order i to catch order consolidating or cancellation. We can also define $T \triangleq \max_{i\in[N]}\{t_i + \ell_i\}$ as the time horizon of

the instance $\mathcal{I}$, since all orders must have been dispatched by that time. All orders will be realized in an online fashion, and both N and T are unknown to the system (i.e., decision-maker).

3.1. Core Model

We refer to the special case of this model where all lifespans are identical as our *core model*. All of our analysis will focus on the core model, until the extensions in Sections 6 and 7. We next justify the three key assumptions in our core model. Note that all these assumptions are also made for practical reasons in Wang et al. (2024) (see Section 3.2 for a detailed comparison).

First, we assume that all lifespans are identical. This assumption is mainly driven by business constraints (e.g., all one-day shipping orders can be held up to 30 minutes). By re-scaling time, we normalize them all to one, i.e., $\ell_i = 1 \ \forall i \in [N]$. Later on, we relax this assumption and study the setting with heterogeneous lifespans ℓ_i in Section 6.

Second, we assume that the system has a capacity C in the sense that we cannot hold more than C number of orders at any point in time. This assumption is made mainly for practical reasons; see Subsection 3.2 for further discussion on the practical basis of this assumption.

Third, the reward rate h_i is constant over time. This assumption can be reasonably justified, because according to Wang et al. (2024), the interarrival time of two consecutive orders placed by the same person can be well approximated by an Exponential distribution or a mixture of multiple Exponential distributions. In addition, when an order cannot be held too long (e.g., up to 30 minutes), this assumption essentially says that the likelihood of multiple orders occurring is equal across the holding window. We later relax this assumption and study the setting in which the reward rate of holding any order can vary over time in Section 7.

Remark 1. The reward rate is assumed to be deterministic in this paper, either time-invariant (core model) or time-varying (Section 7). This assumption applies to scenarios where the reward process can be modeled as the accumulation of a known, user-specific reward rate over time. For instance, in cloud computing, this corresponds to the price per unit of computation time, such as the price per unit of computation time in cloud computing. In contrast, some applications, such as order consolidation, may involve stochastic reward processes (e.g., multiorder cost savings modeled as jump processes). As explained in Appendix A, such stochastic reward processes can be reformulated as equivalent deterministic reward processes by defining appropriate reward rates, as long as the stochastic reward process for holding each order has known (time-varying) means that are fixed upon arrival and do not depend on any realizations.

We formally define a policy π (that could be online or offline) as follows. Let us first define $\mathcal{H}(t)$ as the set of indices for the orders that are being held in the system at time t, which we refer to as the holding set at time t. In addition, we define $\mathcal{A}(t)$ as the set of orders that are arriving at

time t, i.e., $\mathcal{A}(t) \triangleq \{i | t_i = t\}$. A policy π can hold/dispatch any of those already arrived orders, i.e., $\mathcal{H}(t) \subseteq \{\cup_{\tau \leq t} \mathcal{A}(\tau)\}$ for all t. π is called feasible if it satisfies the following constraints for all t:

$$\mathcal{H}(t) \subseteq \mathcal{H}(t^-) \cup \mathcal{A}(t), \tag{1}$$

$$|\mathcal{H}(t)| \leq C, \tag{2}$$

$$\mathcal{H}(t) \bigcap \left\{ i \in [N] \middle| t_i + 1 \leq t \right\} = \varnothing, \tag{3}$$

where t^- refers to the infinitesimal instant in time right before t. Here, (1) states that at any time t, π can only hold a subset of what it was holding right before time t as well as the newly-arrived orders. This constraint prevents a policy from re-holding an order that was dispatched earlier, primarily motivated by the practice that once an order is dispatched to a warehouse for picking and packing, it cannot be re-held in the virtual pool. (2) is the capacity constraint, and (3) ensures that no order can be held in the system after reaching its deadline. We denote the set of all feasible policies by Π_f. We call a policy online if its decisions on which orders to dispatch do not rely on future information. More formally, its decision at time t only depends on $\left\{ \{\mathcal{A}(t')\}_{t' \in [0,t]}, \{\mathcal{H}(t')\}_{t' \in [0,t)} \right\}$. We also denote by Π_f^{on} ($\subseteq \Pi_f$) the set of all online feasible policies.

In the following we define additional notations that will help us to study and analyze our model. Given any policy and arrival instance of orders, we define d_i as the dispatch time of the order i; i.e., $d_i = \sup_t \left\{ t \in [t_i, t_i + 1) \middle| i \in \mathcal{H}(t) \right\}$, with $d_i = t_i$ if i was never held. With all this in mind, we can formulate the utility of any policy π in a system with capacity C under any instance $\mathcal{I}$, as:

$$U_\pi(\mathcal{I}; C) \triangleq \int_0^T \left(\sum_{i \in \mathcal{H}(t)} h_i \right) dt = \sum_{i \in [N]} h_i (d_i - t_i). \tag{4}$$

Note that both $\mathcal{H}(t)$ and d_i depend on π, and we suppress this dependence for the simplicity of notations.

Let $\mathsf{OPT}(\mathcal{I}; C) \triangleq \max_{\pi \in \Pi_f} U_\pi(\mathcal{I}; C)$ denote the maximum total reward that can be extracted from instance $\mathcal{I}$ by a policy π that can be either offline (i.e., knowing the future) or online, but must still be feasible (i.e., satisfying conditions (1)–(3) above). For ease of notation, we drop the dependence on C and write $U_\pi(\mathcal{I})$ and $\mathsf{OPT}(\mathcal{I})$ as the shortened versions of $U_\pi(\mathcal{I}; C)$ and $\mathsf{OPT}(\mathcal{I}; C)$, respectively, and only make this dependence explicit as needed (e.g., in Section 6.2). We also refer to the optimal offline policy by OPT for convenience. Additionally, let $\arg\max^C$ be the operator that returns the indices (order's number) for the C largest elements in a set (that tie-breaks in favor of larger indices, and returns the entire set if it has fewer than C elements). We also define $S_t^C \triangleq \arg\max_i^C \{h_i | \forall i : t_i \leq t < t_i + 1\}$, which consists of the indices of the (at most) C orders with largest reward rates among all orders that are active at time t.

We further define the following *Skyline* relaxation from OPT as:

$$\mathsf{Skyline}(\mathcal{I}) \triangleq \sum_i h_i \left(\int_{t_i}^{t_i+1} \mathbb{1}[i \in S_t^C] \, dt \right) = \int_0^T \left(\sum_{i \in S_t^C} h_i \right) \, dt.$$

Here, we relax constraint (1), i.e., $\mathsf{Skyline}$ allows re-holding an order that was previously released as long as its deadline has not been reached, whereas OPT cannot re-hold an order. With this in mind, it is not hard to see that $\mathsf{Skyline}$ always holds the (up to) C active orders with largest reward rates. Thus, for any policy π and instance $\mathcal{I}$, we have:

$$U_\pi(\mathcal{I}) \leq \mathsf{OPT}(\mathcal{I}) \leq \mathsf{Skyline}(\mathcal{I}).$$

We use *competitive ratio* as our performance metric when comparing online policies against the optimal offline OPT (or, similarly defined, against the Skyline relaxation $\mathsf{Skyline}$). More formally, we define competitive ratio of an algorithm π w.r.t. OPT as follows:

$$\mathcal{CR}(\pi) \triangleq \inf_{\mathcal{I}} \frac{U_\pi(\mathcal{I})}{\mathsf{OPT}(\mathcal{I})}.$$

We note that $\mathcal{CR}(\pi)$ both provides a guarantee on the fraction of $\mathsf{OPT}(\mathcal{I})$ that is obtained by algorithm π on any instance $\mathcal{I}$, and also indicates that there exists a worst-case instance on which this fraction is exactly $\mathcal{CR}(\pi)$. We also conduct a concise numerical study in Section 8 to evaluate the average performance of proposed algorithms under non-worst-case demand sequences.

3.2. Model Relevance for E-commerce Fulfillment

In this section, we discuss the relevance of our model to the practical challenges faced by an anonymous Chinese online grocery retailer (Wang et al. 2024). We begin by summarizing their order fulfillment process, as illustrated in Wang et al. (2024, Figure 1):

1. **Order Validation:** When a customer places an order, the system initially verifies for any issues, including incomplete addresses or possible fraud, following payment and confirmation.

2. **Order Assignment:** Orders that clear the initial check are assigned to one or more fulfillment centers (FCs). To optimize the assignment and minimize overall fulfillment costs, the e-retailer often utilizes an online algorithm. This process may require splitting multi-item orders (i.e., orders containing multiple items) into separate parcels due to inventory constraints. The primary focus in the fulfillment literature is on designing an effective online algorithm for this step (see, e.g., Acimovic and Graves 2015, Jasin and Sinha 2015, Zhao et al. 2020, Amil et al. 2022, Xie et al. 2024, Ma 2023).

3. **Order Consolidation:** The third step involves consolidating multiple customer orders into a single parcel to reduce shipping costs, as shipping is typically charged per parcel. However, not

all orders can be consolidated; specific criteria must be met. These criteria include the orders being placed by the same customer, having the same delivery address, being placed on the same day, being assigned to the same fulfillment center, and meeting the free shipping requirements. The reward h_i in our model represents the rate at which order i can be consolidated with a future order that satisfies these criteria. At the end of the consolidation period, orders are released from the virtual pools to the FCs for execution, which includes item picking, parcel packing and labeling, and ultimately, delivery to customers.

The problem of interest to both Wang et al. (2024) and us is determining how to make order release decisions in the third step of the fulfillment process described above. Wang et al. (2024) model this problem using a finite-horizon discrete-time Markov decision process, aiming to maximize the expected number of multiorder hits, i.e., orders that can be consolidated with a previous order, as described in the third step above. Their state is represented by the orders in the virtual pool, each associated with a consolidation probability (equivalent to our reward rate h_i) and the duration of its stay. Their action involves deciding which customers' orders to release in each time period. Their model includes several constraints, the most significant of which are:

1. **Maximum Stay Constraint:** Each order can only remain in the pool for a certain maximum duration (equivalent to our ℓ).

2. **Pool Size Constraint:** There is a limit on the total number of orders that can be held in the pool at any time (equivalent to our C). This constraint is mainly for practical reasons. In an e-commerce environment, especially with large order volumes, this constraint serves as a practical proxy to ensure a steady rate of order dispatch from the virtual pool for warehouse picking, preventing later congestion in warehouse operations and outbound transportation. Without this constraint, all orders would be held for their maximum duration, which, as discussed previously, is unacceptable. Such a capacity constraint is also evident in other applications like cloud computing, as highlighted in Subsection 1.1.

3. **Maximum Outflow Constraint:** A penalty is incurred for releasing more orders from the pool than allowed by the order-processing capacity constraint in any given time period.

Our problem can be interpreted as a parsimonious formulation of the key trade-off studied in Wang et al. (2024); namely, which orders to release given constraints 1. and 2. above. We ignore the penalty associated with constraint 3., and consider the problem in terms of which orders to hold instead of which to release. We note that our objective (deterministically collecting reward h_i per unit of time order i is held) is identical in expectation to the multiorder-hits objective of Wang et al. (2024); see Appendix A for a rigorous explanation. However, we do not take an expectation over the randomness in order arrivals, instead treating them as adversarial. The rationale is twofold: first, an adversarial arrival model allows us to distill the key trade-off in the problem (see Figure 1)

and develop interpretable policies; second, the performance guarantees under our adversarial model hold robustly, even without assuming the correctness of the stochastic arrival model.

3.3. Index-Based Policies

In this section, we introduce the family of index-based policies. Each policy in this family is associated with an index (or score) function that measures the value of an order at a certain point of time, and the policy compares the indices among candidate orders to make dispatch decisions. Let $f_z : \mathbb{R} \to \mathbb{R}$ be an index function associated with an order $z \triangleq (t, h)$ defined by its height h and arrival time t, and $f_z(t')$ represents the index value of the order z at any time $t' \in \mathbb{R}$.

Definition 1 (**Index-Based Policy**). An index-based policy with index function set $\{f_z(.)\}_{z \in \mathbb{R}_+^2}$, at any time t, holds the following orders (if any) and dispatches the rest:

$$\mathcal{H}(t) = \arg\overset{C}{\max}\{f_{z_i}(t) | i \in \mathcal{H}(t^-) \cup \mathcal{A}(t) \setminus \mathcal{A}(t-1)\}.$$

In words, at each time, an index-based policy first dispatches the orders that have reached their deadline and then keeps the (at most) C orders with highest index values at that time, among the remaining active orders, and dispatches the rest (if there are more than C of them). We also want to highlight that, any index-based policy would be a feasible online policy, resulted simply from its definition above.

To motivate our design of index functions, recall the following discussion from the Introduction. Given an order i that is currently being held which arrived at time t_i with height h_i, and a newly-arrived order at the current time $t \in (t_i, t_i + 1)$ with height h, it is always better to hold the new order if $h \geq h_i$, and always better to continue to hold order i if $h_i(1 + t_i - t) \geq h$. Therefore, we focus on the regime where $h_i(1 + t_i - t) \leq h \leq h_i$, in which case we have a trade-off. One sensible approach is to first analyze the two extreme index functions for making this trade-off:

- the extreme where the index is the height of the order, i.e. $f_z(t') = h$, implying that the algorithm would always dispatch the newly-arriving order (with lower height) when there is a trade-off;
- the extreme where the index is the remaining area of the order, i.e. $f_z(t') = h(1 + t - t')$, implying that the algorithm would always dispatch the currently-held order (with lower remaining area) when there is a trade-off.

These warm-up algorithms are analyzed in Subsections 3.4 and 3.5 respectively. Afterward we move toward a more sophisticated family of algorithms (called $\pi^{\mathcal{B}}$) in Section 4, which aim to balance between the two extremes.

3.4. Policy that Holds the Orders with Largest Heights

Our first algorithm always holds the (up to) C orders with the highest reward rates.

Definition 2 (π^{h}**: Hold Highest Heights)**. π^{h} is the index-based policy with index function $f_z(t') = h$ for all $t' \in \mathbb{R}$.

π^{h} is both intuitive and efficient, making it appealing and straightforward to implement in practice. For example, as previously mentioned, it is utilized in Vast.ai's interruptible instance rental model. It achieves at least $\frac{1}{2}$ competitive ratio against the optimal offline OPT, as well as the stronger benchmark $\mathsf{Skyline}$.

Lemma 1. *For any instance $\mathcal{I}$, we have*

$$2\ U_{\pi^{\mathsf{h}}}(\mathcal{I}) \geq \mathsf{Skyline}(\mathcal{I}) \geq \mathsf{OPT}(\mathcal{I}).$$

We next show that the competitive ratio of policy π^{h} is exactly $\frac{1}{2}$, i.e. π^{h} can earn no better than 1/2 of the optimum, in a worst-case instance.

Proposition 1. *The competitive ratio of policy π^{h} is*

$$\mathcal{CR}(\pi^{\mathsf{h}}) = \frac{1}{2}.$$

3.5. Policy that Holds the Orders with Largest Remaining Areas

Our second algorithm always holds the (up to) C orders with the largest remaining areas.

Definition 3 (π^{ra}**: Hold Largest Remaining Areas)**. π^{ra} is the index-based policy with index function $f_z(t') = h(t+1-t')$ for all $t' \in [t, t+1]$ and is equal to zero outside of this interval.

π^{ra} is also intuitive and simple to implement, and has a higher competitive ratio than π^{h}.

Proposition 2. *The competitive ratio of policy π^{ra} is*

$$\mathcal{CR}(\pi^{\mathsf{ra}}) = 1 - \frac{1}{e} \geq 0.63.$$

4. Framework for Designing and Analyzing a Family of Index-based Policies

In this section, we introduce and analyze a new family of index functions, which aim to make more balanced decisions than the two extreme policies $\pi^{\mathsf{h}}, \pi^{\mathsf{ra}}$.

4.1. General Framework based on Continuous Partitioning and Base Curves

We first introduce the notion of "continuous partitioning" of $\mathbb{R}^2$. Define a collection of curves $\{\mathcal{B}_r(.)\}_{r\in\mathbb{R}}$ (called "Base Curves"), where $\mathcal{B}_r(.) : \mathbb{R} \to \mathbb{R}$. We say they are a continuous partitioning of $\mathbb{R}^2$ if:

1. for any $r \in \mathbb{R}$, $\mathcal{B}_r(.)$ is a continuous function with $\mathcal{B}_r(0) = r$;

2. each point $z = (t, h) \in \mathbb{R}^2$ falls on exactly one of these functions, whose subscript is denoted by r_z; in other words, r_z is the unique value for which $h = \mathcal{B}_{r_z}(t)$.

One such example is $\mathcal{B}_r(t) \triangleq re^{-t}$, and it is easy to verify that the aforementioned two properties are satisfied. Note that $\mathcal{B}_r(.)$ does not have to be decreasing, and it can even be non-monotonic. For convenience, we sometimes refer to the continuous partitioning $\{\mathcal{B}_r(.)\}_{r\in\mathbb{R}}$ as just $\mathcal{B}$.

The properties of continuous partitioning guarantee that the base curves must have a parallel structure, in the sense that if a curve is higher than another one at some time t, then it must be higher at all times. Combining this observation with the definition that $\mathcal{B}_r(0) = r$, we have

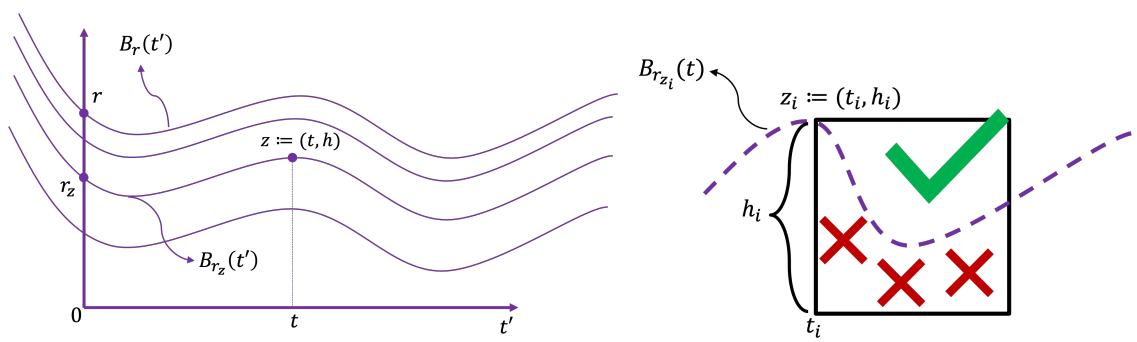
$$r \geq r' \iff \mathcal{B}_r(t) \geq \mathcal{B}_{r'}(t) \ \forall t, \tag{5}$$

a fact to be used in our analysis.

We now introduce a family of index-based algorithms associated with $\mathcal{B}$.

Definition 4 ($\pi^{\mathcal{B}}$**: Index-Based Policy with Base** $\mathcal{B}$). $\pi^{\mathcal{B}}$ is the index-based policy with index function $f_z(t') = \mathcal{B}_{r_z}(t')$ for all $t' \in \mathbb{R}$.

We illustrate these concepts in Figure 2.



(a) An example of continuous partitioning (b) $\pi^{\mathcal{B}}$: corresponding index policy with base $\mathcal{B}$

Figure 2 An illustration of continuous partitioning and index-based policy.

We next define two key parameters associated with $\mathcal{B}$:

$$\gamma_o \triangleq \inf_{z=(t,h)\in\mathbb{R}_+^2,\ t'\in(t,t+1)} \left\{ \frac{\int_t^{t'} \mathcal{B}_{r_z}(t'')\ dt''}{h(t'-t)} \right\}, \tag{6}$$

$$\gamma_a \triangleq \inf_{z=(t,h)\in\mathbb{R}_+^2} \left\{ \inf_{t'\in(t,t+1)} \left\{ \frac{h(t'-t)}{\int_t^{t'} \mathcal{B}_{r_z}(t'')\ dt''} \right\}, \frac{h}{\int_t^{t+2} \mathcal{B}_{r_z}(t'')\ dt''} \right\}. \tag{7}$$

Note that if all the base curves $\mathcal{B}_r(.)$ are non-increasing, then the inner inf operator in (7) would be equal to one (minimized when t' is arbitrarily close to t, recalling that $\mathcal{B}_{r_z}(t) = h$), greatly simplifying the expression of γ_a. In fact, the exponential base curves to be discussed in Section 4.3

satisfy this property. Regarding the calculation of these two parameters, we are able to explicitly calculate them in the special case of exponential base curves (as stated in Lemma 5). Additionally, for other potential base curves with "good" properties (e.g., Lipschitz continuous in the domain of the problem), we can efficiently compute them numerically with a grid search, because the dimension of the minimization problems in (6) and (7) are three.

We now state the key result of the paper, which provides a way to explicitly calculate a lower bound on the competitive ratio of $\pi^{\mathcal{B}}$ for any $\mathcal{B}$. Here, the lower bound involves γ_a and γ_o.

THEOREM 1. *For any base $\mathcal{B}$, the competitive ratio of policy $\pi^{\mathcal{B}}$ is at least $\gamma_a\gamma_o$:*

$$U_{\pi^{\mathcal{B}}}(\mathcal{I}) \geq \gamma_a\gamma_o \mathsf{OPT}(\mathcal{I}) \quad \forall \mathcal{I}.$$

One key component in our argument is to decouple the entanglement between $\pi^{\mathcal{B}}$ and the optimal offline OPT, by breaking their comparison into three separate parts that we study in Lemmas 2, 3, and 4 below. More specifically, Lemma 2 compares the utility under policy $\pi^{\mathcal{B}}$ with some variant of the utility (called "quasi-utility") under an adjusted version of policy $\pi^{\mathcal{B}}$ (called "extended" policy $\tilde{\pi}$) in the "Policy space". Lemma 4 does a similar comparison in the "Optimal-offline space" by comparing the quasi-utility under the optimal offline with its actual utility. The construction of γ_a and γ_o come naively from the comparisons. Last but not least, Lemma 3 bridges the two (disentangled) worlds by relating the quasi-utility under the optimal offline OPT to that under the extended policy $\tilde{\pi}$.

We also hope to highlight that the ratios of both Lemmas 2 and 4 (i.e., γ_a, γ_o) only rely on the geometrical structure of the base $\mathcal{B}$. Accordingly, this framework can transform the problem of finding improved competitive ratios for an online decision-making problem to an offline geometry problem in which we aim to find the best (w.r.t. the problem's specifics) continuous partitioning of $\mathbb{R}^2$ that returns a higher multiplicative factor of $\gamma_a\gamma_o$.

4.2. Proof of Theorem 1

In this subsection, we prove Theorem 1. As discussed earlier, we break the proof into three parts.

We first introduce a key concept - an imaginary policy $\tilde{\pi}$ (called "extended" policy of π) that is built upon policy π. It is possibly infeasible (both capacity-wise and deadline-wise) and its usage is solely for the purpose of analyzing the competitive ratio between the original policy π and optimal offline OPT. Given a feasible policy π, we define its extended policy $\tilde{\pi} \triangleq \tilde{\pi}(\pi)$ as follows (we usually just write $\tilde{\pi}$ for brevity).

DEFINITION 5 ($\tilde{\pi}$**: Extended Policy of** π**).** Consider a policy π, which dispatches order i at time $d_i, \forall i \in [N]$. Then, we define the dispatch times under $\tilde{\pi}$ (the extended policy of π) as follows. For any order i, if $d_i < t_i + 1$, $\tilde{\pi}$ will also dispatch this order at time d_i; otherwise (if $d_i = t_i + 1$), $\tilde{\pi}$

will hold the order for an additional unit of time. More formally, let $\tilde{d}_i$ denote the dispatch time of order i under $\tilde{\pi}$. Then,

$$\tilde{d}_i = \begin{cases} d_i & \text{if } d_i < t_i + 1; \\ d_i + 1 & \text{if } d_i = t_i + 1. \end{cases}$$

We also introduce a notion of "quasi-utility" for any policy π defined by dispatch times $\{d_i\}_{i\in[N]}$, denoted by $V_{\pi,\mathcal{B}}(\mathcal{I})$, similar to the definition of utility $U_\pi(\mathcal{I})$ in (4) with the difference that the reward is accumulated at the rate of the base $\mathcal{B}$ corresponding to each order instead of its height. We emphasize that this definition is valid even for policies such as $\tilde{\pi}$ which may not be feasible. In particular, if $\tilde{d}_i = t_i + 2$ for some order i, then $\tilde{\pi}$ would collect reward from i at the rate given by the base curve for a duration of length 2.

Definition 6 ($V_{\pi,\mathcal{B}}$**: Quasi-Utility of** π **w.r.t. base** $\mathcal{B}$**).** The quasi-utility of a policy π w.r.t. base $\mathcal{B}$ under instance $\mathcal{I}$ is defined as:

$$V_{\pi,\mathcal{B}}(\mathcal{I}) \triangleq \sum_{i\in[N]} \left(\int_{t_i}^{d_i} \mathcal{B}_{r_{z_i}}(t)\ dt \right).$$

Finally, we think of OPT as a policy that dispatches each order i at time d_i^*. We are now ready to begin our analysis, which compares the true and quasi-utilities of policies $\pi^{\mathcal{B}}$, $\widetilde{\pi^{\mathcal{B}}}$ (the extended policy of $\pi^{\mathcal{B}}$), and OPT.

The following lemma first compares the actual utility of index-based policy $\pi^{\mathcal{B}}$ with the quasi-utility of its extended policy $\widetilde{\pi^{\mathcal{B}}}$.

Lemma 2. *For any instance $\mathcal{I}$ and base $\mathcal{B}$, we have*

$$U_{\pi^{\mathcal{B}}}(\mathcal{I}) \geq \gamma_a V_{\widetilde{\pi^{\mathcal{B}}},\mathcal{B}}(\mathcal{I}).$$

Our next lemma compares the quasi-utility of the extended policy $\widetilde{\pi^{\mathcal{B}}}$ with that of OPT.

Lemma 3. *For any instance $\mathcal{I}$ and base $\mathcal{B}$, we have*

$$V_{\widetilde{\pi^{\mathcal{B}}},\mathcal{B}}(\mathcal{I}) \geq V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}). \tag{8}$$

Intuitively, Lemma 3 says that a policy which keeps the orders with the highest base curves collects at least as much quasi-utility as any other policy, including OPT, if it has the further privilege of "extending" the dispatch time of any order that is being held and reaches its original deadline. Here, we provide a proof sketch for this key idea. We first change the order of summation with integration and show that the summation of the base curve values for the orders that $\widetilde{\pi^{\mathcal{B}}}$ is holding is at least that of the orders that OPT is holding at any time t:

$$\sum_{i\in\mathcal{H}_{\widetilde{\pi^{\mathcal{B}}}}(t)} \mathcal{B}_{r_{z_i}}(t) \geq \sum_{i\in\mathcal{H}_{\mathsf{OPT}}(t)} \mathcal{B}_{r_{z_i}}(t).$$

We then note that, at any time t, any order that is being held by $\pi^{\mathcal{B}}$ is also being held by $\widetilde{\pi^{\mathcal{B}}}$. We next prove a claim that for any order $z=(t,h)$, if its base value $\mathcal{B}_{r_z}(t')$ at some time $t'\in[t,t+1)$ is lower than all the base values of the orders that $\pi^{\mathcal{B}}$ is holding at t', then $\mathcal{B}_{r_z}(t'')$ must be lower than each of the C largest base values that $\widetilde{\pi^{\mathcal{B}}}$ is holding at that time t'', for all $t''\in[t',t'+1)$. Finally, by partitioning the set of orders that OPT is holding at time t into two sets - its intersection with the ones that $\pi^{\mathcal{B}}$ is holding as well as its difference set - and using the result of the claim, we show that each of the C largest base values of the orders that $\widetilde{\pi^{\mathcal{B}}}$ holds is (weakly) larger than each of the base values in the difference set.

Our final lemma compares the quasi-utility of OPT with the actual utility of OPT.

Lemma 4. *For any instance $\mathcal{I}$ and base $\mathcal{B}$, we have*

$$V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I})\geq\gamma_o\mathsf{OPT}(\mathcal{I}).$$

4.3. Exponential Base Curves

In this subsection, we analyze a specific family of base curves that satisfy the two properties of continuous partitioning, and explicitly calculate the corresponding γ_a and γ_o.

Define the following family of exponential base curves w.r.t. the parameter $\alpha\in\mathbb{R}$:

$$\forall r,t\in\mathbb{R},\quad \mathcal{B}_r^{\mathsf{exp}(\alpha)}(t)=re^{-\alpha t}. \tag{9}$$

It is easy to see that $r_z=he^{\alpha t}$. We call $\mathcal{B}^{\mathsf{exp}(\alpha)}$ the *exponential base*, and denote by $\pi^{\mathsf{exp}(\alpha)}$ its associated base policy (i.e., the policy $\pi^{\mathcal{B}}$ in Definition 4 with $\mathcal{B}=\mathcal{B}^{\mathsf{exp}(\alpha)}$).

We now focus on analyzing the exponential base $\mathcal{B}^{\mathsf{exp}(\alpha)}$. The following lemma explicitly computes the γ_a and γ_o associated with the exponential base $\mathcal{B}^{\mathsf{exp}(\alpha)}$.

Lemma 5. *For any $\alpha\geq 0$, the γ_a and γ_o associated with the exponential base $\mathcal{B}^{\mathsf{exp}(\alpha)}$ are*

$$\gamma_a=\min\left\{1,\ \frac{\alpha}{1-e^{-2\alpha}}\right\},\quad \gamma_o=\frac{1-e^{-\alpha}}{\alpha}.$$

Combining Lemma 5 with Theorem 1, we derive a lower bound on the competitive ratio of $\pi^{\mathsf{exp}(\alpha)}$, which equals $\gamma_a\gamma_o=\min\left\{\frac{1-e^{-\alpha}}{\alpha},\ \frac{1-e^{-\alpha}}{1-e^{-2\alpha}}\right\}$.

We now determine the parameter α that maximizes this quantity $\gamma_a\gamma_o$. Note that the function $\frac{1-e^{-\alpha}}{\alpha}$ is a decreasing function of α, whereas $\frac{1-e^{-\alpha}}{1-e^{-2\alpha}}$ is an increasing function of α. Thus, in order to maximize their minimum, we should equalize the two terms, which is achieved at the unique positive value α^* satisfying $\alpha^*=1-e^{-2\alpha^*}$. This optimized parameter α^* can be expressed as

$$\alpha^*\triangleq 1+\frac{W_0(-2e^{-2})}{2}\approx 0.7968, \tag{10}$$

where $W_0(.)$ is the principal branch of the Lambert W function (cf. Corless et al. 1996).

Under this choice of α^*, we get a competitive ratio lower bound of $\frac{1-e^{-\alpha^*}}{\alpha^*}$, which exceeds 0.68. We summarize our result in the following theorem, which also says that the analysis for this algorithm $\pi^{\mathsf{exp}(\alpha^*)}$ is tight.

THEOREM 2. *The competitive ratio $\mathcal{CR}$ of policy $\pi^{\mathsf{exp}(\alpha^*)}$ satisfies*

$$\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)}) = \frac{1-e^{-\alpha^*}}{\alpha^*} \geq 0.68.$$

5. Upper Bounds

In Sections 3 and 4, we considered specific index-based policies and provided tight analyses for their competitive ratios, under our core model. In this section we derive upper bounds on the competitive ratio of *any* algorithm, which provide some benchmarks for how close our algorithms are to theoretically optimal under our core model.

First, we show that the competitive ratio of the best algorithm we derived, the exponential-index-based policy with a competitive ratio exceeding 0.68, beats what can be obtained using the looser Skyline relaxation. In fact, even the optimal offline policy OPT cannot earn more reward than $\frac{2}{3}\mathsf{Skyline}$. This shows why we had to consider the more sophisticated behavior of OPT in Section 4 to beat 2/3 (compared to, e.g., the simpler Lemma 1 in Section 3).

PROPOSITION 3 (OPT **vs.** $\mathsf{Skyline}$). *The worst-case ratio of $\frac{\mathsf{OPT}(\mathcal{I})}{\mathsf{Skyline}(\mathcal{I})}$ among all $\mathcal{I}$ is upper bounded by $\frac{2}{3}$. This fact holds under any amount of pool capacity C.*

Next we show that even comparing to OPT, algorithms that are deterministic (which all of our index-based policies satisfy) cannot have a competitive ratio exceeding 0.84.

PROPOSITION 4 (**Deterministic Algorithms vs.** OPT). *When $C=1$, the competitive ratio of any feasible deterministic online policy $\pi \in \Pi_f^{on}$ compared to optimal offline is upper bounded by*

$$\mathcal{CR}(\pi) \leq x^*(2-x^*) \leq 0.84,$$

where $x^* \approx 0.597$ is the unique solution to $2x^2(2-x)=1$ on $[0,1]$.

Finally, we briefly discuss randomized algorithms, which specify a distribution of deterministic algorithms to use. Their utility is defined as the expected utility of a deterministic algorithm drawn from this distribution. In the definition of competitive ratio, the worst-case instance $\mathcal{I}$ must be chosen without knowing the deterministic algorithm drawn (assuming an oblivious adversary, as standard), which is why randomized algorithms can sometimes achieve better competitive ratios than their deterministic counterparts. Nonetheless, we modify the construction from Proposition 4 to show that randomized algorithms cannot do too much better.

PROPOSITION 5 **(Randomized Algorithms vs. OPT)**. *When $C=1$, the competitive ratio of any feasible randomized online policy compared to optimal offline is upper bounded by*

$$\frac{1+x^*(2-x^*)}{2} \leq 0.92.$$

6. Extension: Heterogeneous Lifespans

In this section, we relax the assumption that all orders have the same lifespan. We assume that each order $i \in [N]$ has lifespan $\ell_i \in [1, L]$ for some $L \geq 1$, with L being known. We note that the lifespan feasibility constraint Equation (3) is now transformed to

$$\mathcal{H}(t) \bigcap \left\{ i \in [N] \middle| t_i + \ell_i \leq t \right\} = \varnothing.$$

One could naively apply the algorithms designed from the core model with homogeneous lifespans earlier, but this approach would lose a factor of L in the competitive ratio. In Subsection 6.1, we provide a more elaborate design, which enables any algorithm studied for the core model to handle heterogeneous lifespans while losing a factor of at most $\sqrt{L}$ in the competitive ratio. We also show this loss to be unavoidable; namely, the competitive ratio of any deterministic algorithm under heterogeneous lifespans is upper bounded by $1/\sqrt{L}$. Later in Subsection 6.2, we prove that the competitive ratio can be significantly improved to $\Omega\left(1/\log(L)\right)$ under a large-capacity regime where $C = \Omega\left(\log(L)\right)$.

6.1. Regime: General C

In this subsection, we do not make any assumptions on the capacity and allow it to be any $C \geq 1$. Let us start with the following definition of transforming an original instance $\mathcal{I}$ with heterogeneous lifespans into a transformed instances $\widetilde{\mathcal{I}}$ with homogeneous lifespans.

DEFINITION 7 **($\widetilde{\mathcal{I}}$: Transformed Instance)**. Given an instance $\mathcal{I} = \left(N, \{(t_i, h_i, \ell_i)\}_{i\in[N]}\right)$, where all lifespans are in $[1, L]$, we construct the following transformed instance. For each order i,

1. if $\ell_i \geq \sqrt{L}$, we truncate its lifespan to $\widetilde{\ell_i} = \sqrt{L}$ and keep other parameters the same, i.e., $\widetilde{h_i} = h_i$ and $\widetilde{t_i} = t_i$;
2. if $\ell_i < \sqrt{L}$, we stretch the order to have lifespan $\sqrt{L}$ while preserving its total area and arrival time, i.e., $\widetilde{\ell_i} = \sqrt{L} > \ell_i$, $\widetilde{h_i} = \frac{h_i \ell_i}{\sqrt{L}} < h_i$, and $\widetilde{t_i} = t_i$.

We define $\widetilde{\mathcal{I}} \triangleq \left(N, \{(\widetilde{t_i}, \widetilde{h_i}, \widetilde{\ell_i})\}_{i\in[N]}\right)$, and let $\widetilde{z_i}$ be the order in $\widetilde{\mathcal{I}}$ that corresponds to order z_i in $\mathcal{I}$ for all $i \in [N]$.

Note that all the order arrival times are the same between $\mathcal{I}$ and $\widetilde{\mathcal{I}}$. Whenever an order arrives in the original instance $\mathcal{I}$, since its lifespan is known at arrival, we can compute its corresponding order in $\widetilde{\mathcal{I}}$ and construct $\widetilde{\mathcal{I}}$ in real-time.

Now consider any policy $\widetilde{\pi}$ with competitive ratio $\mathcal{CR}(\widetilde{\pi})$ under the homogeneous lifespan setting where all lifespans are equal to $\sqrt{L}$. For simplicity, we can just think about $\widetilde{\pi} \in \left\{\pi^{\mathsf{h}}, \pi^{\mathsf{ra}}, \pi^{\mathsf{exp}(\alpha^*)}\right\}$ with known competitive ratios. Note that after scaling the $\sqrt{L}$ lifespan to 1, we get all the same competitive ratio results as before.

The intuition behind the construction in Definition 7 is that an algorithm can (easily) collect at least as much reward on the original instance $\mathcal{I}$ compared to the transformed instance $\widetilde{\mathcal{I}}$; meanwhile, the optimal reward on $\mathcal{I}$ compared to $\widetilde{\mathcal{I}}$ is greater by a factor of at most $\sqrt{L}$. Our proof is heavily based on this intuition. Now we formalize it, starting from the construction of an algorithm $\pi^{\mathsf{red}}(\widetilde{\pi})$ for heterogeneous lifespans based on the algorithm $\widetilde{\pi}$ for homogeneous lifespans.

Definition 8 ($\pi^{\mathsf{red}}(\widetilde{\pi})$**: Reduction Policy)**. Given any heterogeneous instance $\mathcal{I}$ and a policy $\widetilde{\pi}$ with competitive ratio $\mathcal{CR}(\widetilde{\pi})$ under the homogeneous lifespan setting, let $\widetilde{\mathcal{I}}$ be the homogeneous transformed instance of $\mathcal{I}$ and we construct $\pi^{\mathsf{red}}(\widetilde{\pi})$ as follows. For any order i at its arrival time, we feed the arrival of $\widetilde{z}_i$ into $\widetilde{\pi}$ and specify its dispatch time under $\pi^{\mathsf{red}}(\widetilde{\pi})$ as: $d_i^{\pi^{\mathsf{red}}(\widetilde{\pi})} = \min(d_i^{\widetilde{\pi}}, t_i + \ell_i)$.

Note that the second argument $t_i + \ell_i$ in the min operator (in the definition of $d_i^{\pi^{\mathsf{red}}(\widetilde{\pi})}$) can only be binding if $\ell_i < \sqrt{L}$, in which case the lifespan of i in the transformed instance is longer than its original lifespan. In the rest of this section as well as its proofs in the appendix, we simply use $\widetilde{\mathcal{CR}} \triangleq \mathcal{CR}(\widetilde{\pi})$ and $\pi^{\mathsf{red}} \triangleq \pi^{\mathsf{red}}(\widetilde{\pi})$ for notational brevity.

Combining the way we determined the dispatch times under π^{red} with the fact that $\widetilde{\pi}$ is an online feasible policy that is running on $\widetilde{\mathcal{I}}$, we can conclude that: (i) π^{red} is also an online policy which never re-holds orders that it has dispatched, (ii) π^{red} satisfies the capacity constraint, because we only reduce the dispatch times compared to $\widetilde{\pi}$, and (iii) π^{red} satisfies the lifespan constraint, because we ensure that the dispatch time for each order is no later that its deadline. In other words, we have $\pi^{\mathsf{red}} \in \Pi_f^{on}$, simply by construction.

We are now ready to state the main result of this subsection.

Theorem 3. *For any instance $\mathcal{I}$ and policy $\widetilde{\pi}$ with competitive ratio of $\widetilde{\mathcal{CR}}$ under the homogeneous lifespan setting, we have*

$$U_{\pi^{\mathsf{red}}}(\mathcal{I}) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L}} \mathsf{OPT}(\mathcal{I}).$$

It follows that $\mathcal{CR}(\pi^{\mathsf{red}}) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L}}$.

We remark that the policy $\widetilde{\pi}$ can be any of the policies we introduced in the preceding. For instance, we can choose $\widetilde{\pi} = \pi^{\mathsf{exp}(\alpha^*)}$, which gives us $\widetilde{\mathcal{CR}} = \frac{1-e^{-\alpha^*}}{\alpha^*} \geq 0.68$.

The previous result provides a lower bound on the competitive ratio with a decay rate of $O\left(\frac{1}{\sqrt{L}}\right)$. We next show that this decay rate is the best that we can hope for, when $C = 1$.

PROPOSITION 6. *When $C=1$, the competitive ratio of any deterministic feasible online policy $\pi \in \Pi_f^{on}$ compared to optimal offline is upper bounded by*

$$\mathcal{CR}(\pi) \leq \frac{1}{\sqrt{L}}.$$

6.2. Regime: $C = \Omega(\log(L))$

In this subsection, we show that if the capacity satisfies $C=\Omega(\log(L))$, then the competitive ratio can be improved dramatically from $\Omega\left(\frac{1}{\sqrt{L}}\right)$ to $\Omega\left(\frac{1}{\log(L)}\right)$. In order to show this, we divide an instance into multiple sub-instances and run a policy with a good competitive ratio on each of the sub-instances. The final policy is then the aggregate policy of all these sub-policies, each running on their corresponding sub-instance, which is constructed from the original instance in an online fashion.

Let us describe how to divide an instance $\mathcal{I}$ whose orders have lifespans in the range of $[1,L]$ into sub-instances $\{\mathcal{I}_k\}_{k\in[K]}$, for some $K \in [C]$ to be specified. $\mathcal{I}_k$ is the instance consisting of all orders in $\mathcal{I}$ whose lifespan lies in the interval $[L^{\frac{k-1}{K}}, L^{\frac{k}{K}}]$, for all $k \in [K] \triangleq \{1,2,...,K\}$. We define the capacity to be $\lfloor \frac{C}{K} \rfloor$ on each sub-instance. Notice that for each of these sub-instances, the ratio of the largest to the smallest possible lifespan is exactly $L^{\frac{1}{K}}$. For each $k \in [K]$, let π^k be a copy of the policy $\pi^{\mathsf{red}}(\widetilde{\pi})$ from Subsection 6.1, whose competitive ratio is at least $\frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}}$ (e.g., if $\widetilde{\pi} = \pi^{\mathsf{exp}(\alpha^*)}$, then $\widetilde{\mathcal{CR}} \geq 0.68$) for the sub-instance $\mathcal{I}_k$ whose orders possess lifespans only in the interval $[L^{\frac{k-1}{K}}, L^{\frac{k}{K}}]$. Note that as orders in the instance $\mathcal{I}$ arrive to the system in real-time, we can route these orders to the sub-instances $\{\mathcal{I}_k\}_{k\in[K]}$ based on its lifespan length.

We now describe our proposed policy.

DEFINITION 9 (π_K^{div} **: Divider Policy**). π_K^{div} has a total capacity of C and divides it into K categories, each consisting of $\lfloor \frac{C}{K} \rfloor$ units of capacity. As the orders in $\mathcal{I}$ arrive over time, for all $k \in [K]$, π_K^{div} constructs $\mathcal{I}_k$ in an online fashion, and with its k-th category of capacity (whose size is $\lfloor \frac{C}{K} \rfloor$), π_K^{div} emulates the exact behavior of running π^k on $\mathcal{I}_k$.

The next lemma provides a lower bound on the competitive ratio of policy π_K^{div}.

LEMMA 6. *For any instance $\mathcal{I}$ and $K \in [C]$, we have*

$$U_{\pi_K^{\mathsf{div}}}(\mathcal{I}) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \frac{\lfloor \frac{C}{K} \rfloor}{C} \mathsf{OPT}(\mathcal{I}).$$

It follows that $\mathcal{CR}(\pi_K^{\mathsf{div}}) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \frac{\lfloor \frac{C}{K} \rfloor}{C}$.

We note that for each sub-instance, the optimum offline with $\lfloor \frac{C}{K} \rfloor$ capacity is at least $\lfloor \frac{C}{K} \rfloor / C$ times the optimum offline with C capacity. This explains the additional factor of $\lfloor \frac{C}{K} \rfloor / C$ in the competitive ratio. Intuitively, the worst case for this divider policy arises when all orders in the

original instance $\mathcal{I}$ fall under the same sub-instance $\mathcal{I}_k$ and we have unnecessarily shrunk our capacity by a factor of K.

Now, let K^* be the choice of K that maximizes the lower bound provided in Lemma 6, i.e.,

$$K^* \triangleq \arg\max_{K\in[C]} \left\{ \frac{\lfloor \frac{C}{K} \rfloor}{\sqrt{L^{\frac{1}{K}}}} \right\}.$$

We next state the main theorem of this subsection: the competitive ratio of $\pi^{\mathsf{div}}_{K^*}$ is significantly improved under a large-capacity regime.

THEOREM 4. *Suppose $C = \Omega(\log(L))$. Then, we have*

$$\mathcal{CR}\left(\pi^{\mathsf{div}}_{K^*}\right) = \Omega\left(\frac{1}{\log(L)}\right).$$

7. Extension: Time-Varying Reward Rates

In this section, we relax the assumption of constant reward rates. Recall that in the core model, the reward collected from order i by algorithm π is equal to $h_i \times (d_i^\pi - t_i)$, where the reward rate of order i remained constant and equal to h_i over the entire time interval $[t_i, d_i^\pi)$. Now we relax this assumption and allow the reward rate to vary over time, i.e., the reward rate of order i at time $t \geq t_i$ is $\mathfrak{h}_i(t)$, where $\mathfrak{h}_i(\cdot)$ is a mapping from $\mathbb{R}_+$ to $\mathbb{R}_+$. This function $\mathfrak{h}_i(t)$ depends on order i and time t, and can be regarded as the derivative of the expected number of orders from customer i during time interval $(t_i, t]$ w.r.t. t. In order to incorporate this change in the definitions of $U_\pi(\mathcal{I})$ and $\mathsf{OPT}(\mathcal{I})$, as well as all of the definitions and results in Section 4, whenever we calculate the reward of some order $z_i = (t_i, \mathfrak{h}_i(.))$ up to some time t', we replace $h_i \times (t' - t_i)$ by $\int_{t_i}^{t'} \mathfrak{h}_i(t'')\, dt''$. Lastly, r_z was defined in Section 4.1 as the index of the base curve that passes through the point (t, h), i.e., the unique r such that $h = \mathcal{B}_r(t)$, for a generic arrival $z = (t, h)$. In this extension, a generic arrival is denoted by $z = (t, \mathfrak{h}(.))$; hence, we change the definition of r_z to be the index of the unique base curve that passes through the point $(t, \mathfrak{h}(t))$, i.e., the unique r such that $\mathfrak{h}(t) = \mathcal{B}_r(t)$. With the integration of these changes, we emphasize that all of our results in Subsection 4.2 with similar proofs still hold, thereby making Theorem 1 to hold even under this extension.

If no conditions are imposed on $\mathfrak{h}_i(t)$, then the competitive ratio can be arbitrarily small, as such reward functions can drop to 0 at any time and capture arbitrarily heterogeneous lifespans as discussed in Section 6. Therefore, we make the following "homogeneity" assumption throughout this section, which says that the reward of order i over time t can be decomposed as the product of two terms:

$$\mathfrak{h}_i(t) = \mathfrak{h}_i(t_i) \cdot \mathfrak{r}(t - t_i) \quad \forall t \geq t_i. \tag{11}$$

Here, $\mathfrak{h}_i(t_i)$ represents the initial value of the reward that depends on i, and $\mathfrak{r} : \mathbb{R}_+ \to \mathbb{R}_+$ satisfying $\mathfrak{r}(0) = 1$ captures the decay rate over time (although we call $\mathfrak{r}$ the decay rate function, we also

allow its value to become greater than one, capturing the situation that customers' likelihood of multiorder may increase over time). It is important to note that this decay rate function $\mathfrak{r}$ is homogeneous across all orders, and it takes as input only the elapsed time $t-t_i$ of order i.

This underlying homogeneity assumption can be justified in practice by customers having the same decay rate in their likelihood of multiorder (or cancel) over time. In other words, while customers may differ in their initial inclination to place multiple orders, their enthusiasm for placing additional orders changes uniformly across all customers as time evolves. Nevertheless, it worth mentioning that if we allow varying decay rates across customers in addition to their different initial likelihood of multiorder, the model would become excessively intricate. This would lead to an overwhelming number of parameters that need to be learned, potentially hindering the effectiveness and practicality of the model.

We focus on the exponential base $\mathcal{B}^{\mathsf{exp}(\alpha)}$ with $\mathcal{B}_r^{\mathsf{exp}(\alpha)}(t)=re^{-\alpha t}$ as defined in (9), to take advantage of the proof techniques we have developed to "analytically" compute the associated γ_a and γ_o. Similar to (6) and (7), we define

$$\gamma_o \triangleq \min_{z\triangleq(t,\mathfrak{h}(.)),\ t'\in[t,t+1)} \left\{ \frac{\int_t^{t'} \mathcal{B}_{r_z}^{\mathsf{exp}(\alpha)}(t'')\ dt''}{\int_t^{t'} \mathfrak{h}(t'')\ dt''} \right\},$$

$$\gamma_a \triangleq \min_{z\triangleq(t,\mathfrak{h}(.))} \left\{ \min_{t'\in[t,t+1)} \left\{ \frac{\int_t^{t'} \mathfrak{h}(t'')\ dt''}{\int_t^{t'} \mathcal{B}_{r_z}^{\mathsf{exp}(\alpha)}(t'')\ dt''} \right\},\ \frac{\int_t^{t+1} \mathfrak{h}(t'')\ dt''}{\int_t^{t+2} \mathcal{B}_{r_z}^{\mathsf{exp}(\alpha)}(t'')\ dt''} \right\}.$$

Under Assumption (11), it is fairly easy to compute the values of γ_o and γ_a. Define

$$R \triangleq \int_0^1 \mathfrak{r}(t'')\ dt'' > 0. \tag{12}$$

Lemma 7. *Under Assumption* (11)*, for any* $\alpha\in\mathbb{R}$*, we have*

$$\gamma_o = \min_{t,\ t'\in[t,t+1)} \left\{ \frac{\int_0^{t'-t} e^{-\alpha t''}\ dt''}{\int_0^{t'-t} \mathfrak{r}(t'')\ dt''} \right\} \le \frac{1-e^{-\alpha}}{\alpha R}, \tag{13}$$

$$\gamma_a = \min\left\{ \min_{t,\ t'\in[t,t+1)} \left\{ \frac{\int_0^{t'-t} \mathfrak{r}(t'')\ dt''}{\int_0^{t'-t} e^{-\alpha t''}\ dt''} \right\},\ \frac{\alpha R}{1-e^{-2\alpha}} \right\} \le \min\left\{1,\ \frac{\alpha R}{1-e^{-2\alpha}}\right\}. \tag{14}$$

In addition, both the inequalities in (13) *and* (14) *will become equality if the function* $\frac{e^{-\alpha t''}}{\mathfrak{r}(t'')}$ *is non-increasing in* t'' *on* $[0,1]$.

If $\alpha=0$, then the expressions involving α in (13) and (14) are understood to denote their limits as $\alpha\to 0$.

As our goal is to maximize $\gamma_a\gamma_o$, Lemma 7 provides a sufficient condition on when the upper bound on $\gamma_a\gamma_o$ can be achieved. Therefore, combining Theorem 1 with the assumption that $\frac{e^{-\alpha t''}}{\mathfrak{r}(t'')}$ is non-increasing in t'' on $[0,1]$, we can conclude that

$$\mathcal{CR}(\pi^{\mathsf{exp}(\alpha)}) \ge \gamma_a\gamma_o = \min\left\{ \frac{1-e^{-\alpha}}{\alpha R},\ \frac{1-e^{-\alpha}}{1-e^{-2\alpha}} \right\}, \tag{15}$$

where $\pi^{\mathsf{exp}(\alpha)}$ is the index-based policy with exponential base $\mathcal{B}^{\mathsf{exp}(\alpha)}$ as defined in Section 4.

Now we want to maximize the value of (15) over α. Note that the first term in the min operator is decreasing in α, and the second term is increasing in α. As such, the maximum value is achieved when the two terms are equal to each other, i.e.,

$$R = \begin{cases} \frac{1-e^{-2\alpha}}{\alpha} & \text{when } \alpha \neq 0; \\ 2 & \text{when } \alpha = 0. \end{cases} \tag{16}$$

We set $\alpha = \alpha^*(R)$ as the unique non-trivial real solution to the equation $\alpha R = 1 - e^{-2\alpha}$ for any $R \neq 2$ (as 0 is always a trivial solution), and set $\alpha = \alpha^*(R) = 0$ for $R = 2$. Equivalently,

$$\alpha^*(R) \triangleq \frac{W_\gamma\left(-\frac{2e^{-\frac{2}{R}}}{R}\right)}{2} + \frac{1}{R}, \quad \gamma \triangleq \begin{cases} 0 & \text{if } R \leq 2; \\ -1 & \text{if } R > 2, \end{cases} \tag{17}$$

where W is the Lambert W function and γ indicates which branch of the Lambert W function to be used (cf. Corless et al. 1996). Note that

$$\alpha^*(R) > 0 \quad \text{if and only if} \quad R < 2,$$

which comes from the fact that the function $\frac{1-e^{-2\alpha}}{\alpha}$ is decreasing in α and reaches 2 at $\alpha = 0$ (more precisely, as $\alpha \to 0$). Thus, we use the principal branch of the Lambert W function if $R \leq 2$, and use the -1 branch otherwise. See an illustration of $\alpha^*(R)$ as a function of R in Figure 3. It follows that

$$\gamma_a = 1, \qquad \gamma_o = \frac{1 - e^{-\alpha^*(R)}}{\alpha^*(R)R} = \frac{1 - e^{-\alpha^*(R)}}{1 - e^{-2\alpha^*(R)}} = \frac{e^{\alpha^*(R)}}{e^{\alpha^*(R)} + 1}. \tag{18}$$

It is important to know that to achieve the values of (18), we only require the function $\frac{e^{-\alpha t''}}{\mathfrak{r}(t'')}$ to be non-increasing for the value $\alpha = \alpha^*(R)$, not all $\alpha \in \mathbb{R}$.

We now discuss the critical assumption (aside from homogeneity) that $\frac{e^{-\alpha^*(R)t''}}{\mathfrak{r}(t'')}$ is non-increasing over $t'' \in [0,1]$. If $\mathfrak{r}$ is differentiable with derivative $\mathfrak{r}'$, then this condition is equivalent to

$$\mathfrak{r}'(t'') + \alpha^*(R)\mathfrak{r}(t'') \geq 0 \quad \forall t'' \in [0,1]. \tag{19}$$

It can then be checked that (19) is satisfied by an exponential function $\mathfrak{r}(t) = e^{-\beta t}$ for any $\beta \in \mathbb{R}$, due to the fact that $\alpha^*(R) > \beta$ (via the definition of $\alpha^*(R)$ in (17) that satisfies (16) and the calculation that $R = \frac{1-e^{-\beta}}{\beta}$ in (12)). This allows us to model, for instance, when customer multiorder probabilities are decaying over time, by setting $\beta > 0$. Note that it reduces to the core model with constant reward rates when $\beta = 0$.

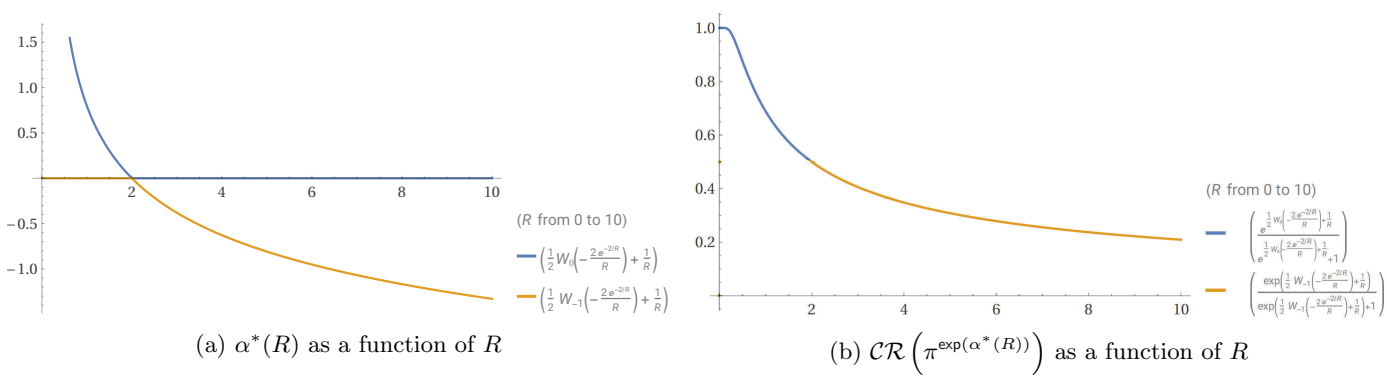
We summarize our findings of this section in the next theorem. Its proof follows similarly from the proof of Theorem 2 and the calculations of γ_a, γ_o as discussed above.

THEOREM 5. *Under homogeneous time-varying reward rates given by a function* $\mathfrak{r}$ *for which* $\frac{e^{-\alpha^*(R)t''}}{\mathfrak{r}(t'')}$ *is non-increasing over* $t'' \in [0,1]$*, with* R *and* $\alpha^*(R)$ *as defined in* (12) *and* (17)*, respectively, we have*

$$\mathcal{CR}\left(\pi^{\exp(\alpha^*(R))}\right) = \gamma_a\gamma_o = \frac{e^{\alpha^*(R)}}{e^{\alpha^*(R)}+1}.$$

See an illustration of $\mathcal{CR}\left(\pi^{\exp(\alpha^*(R))}\right)$ as a function of R in Figure 3. In particular, when $R=1$, we recover the results in Section 4.3 that $\alpha^*(1) \approx 0.7968$ and $\mathcal{CR}\left(\pi^{\exp(\alpha^*(1))}\right) \geq 0.68$. Moreover, $\mathcal{CR}\left(\pi^{\exp(\alpha^*(R))}\right)$ is decreasing in R, starting from 1 when $R=0$. This is because R can be regarded as a parameter for measuring "risk." Indeed, the greater the value of R, the higher the potential regret could be for dispatching an order too early.

As a final note, algorithm $\pi^{\exp(\alpha^*(R))}$ only requires the knowledge of R and the initial reward value $\mathfrak{h}_i(t_i)$ of each order, and the guarantee is robust to the shape of the reward function $\mathfrak{r}(.)$ itself (as long as it satisfies the non-increasing property for $\frac{e^{-\alpha^*(R)t''}}{\mathfrak{r}(t'')}$).



(a) $\alpha^*(R)$ as a function of R

(b) $\mathcal{CR}\left(\pi^{\exp(\alpha^*(R))}\right)$ as a function of R

Figure 3 **Illustrations of $\alpha^*(R)$ and $\mathcal{CR}\left(\pi^{\exp(\alpha^*(R))}\right)$ as functions of R. In Figure (a), the value of $\alpha^*(R)$ follows the blue curve when $R \leq 2$, and follows the orange curve when $R > 2$.**

8. Numerical Experiments

In this section, we conduct a concise numerical study to demonstrate the average performance of the three algorithms (π^{h}, π^{ra} and $\pi^{\exp(\alpha)}$) under the core model where orders have constant reward rates and homogeneous lifespans.

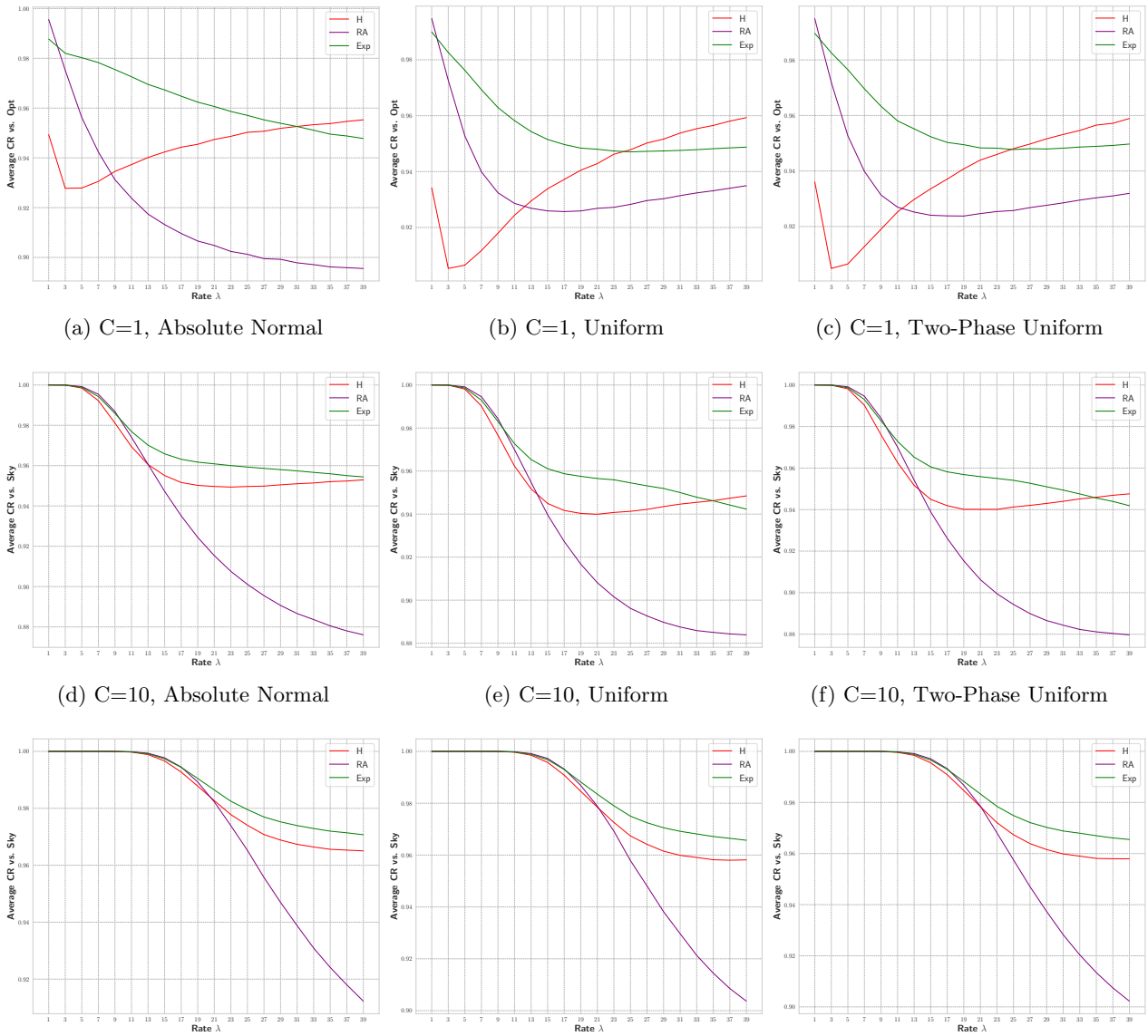
Experimental Setup. We pick capacity $C \in \{1, 10, 20\}$ and fix the time horizon of the problem T and generate $M = 1,000$ instances of an arrival sequence of orders over time from a Poisson process with rate λ. For each order, its reward rate (height) is generated independently from a distribution. We consider three types of distributions: a uniform distribution on [0,1], the absolute value of a standard normal distribution, and a two-phase distribution in which the first half of the heights follows from a uniform distribution on [0,1] and the second half follows from a uniform

distribution on [0,0.5]. Here, the two-phase distribution, with decreasing means, amplifies the challenge in deciding whether to switch to a newly-arrived order with lower height. Note that as C grows, the computational complexity of all of our index-based policies grows linearly (keeping them suitable for practice), whereas the size of the state-space grows exponentially, hindering the ability to compute the optimum offline. In this regard, when $C = 1$, we set $T = 50$ and consider the true optimum offline (OPT) as the performance metric. However, for $C \in \{10, 20\}$, we set $T = 20$ and consider the Skyline relaxation ($\mathsf{Skyline}$) as the performance metric, which is a stronger benchmark with higher value. Regarding performance metric, we use the average (over M instances) of the performance ratio against the corresponding benchmark under the same instance. Under each of the three height distributions, we compute these average performance ratios - for all of the three policies - across various values of λ, and plot the ratio as a function of λ. As shown in Figure 4, the results comply with our expectations formed from the theoretical worst-case ratio analysis for each of the three policies. We now discuss the main findings/takeaways from the numerical results.

Takeaways from Experimental Results. The average ratios of the three algorithms (against either optimum offline or Skyline) - are much higher compared with their theoretic lower bounds, indicating the fact that these simple and interpretable algorithms also perform well in non-adversarial settings. The policy π^{h} is most effective in scenarios where the rate of arrivals is much faster than the capacity to hold orders, because it maximizes the reward rate from each unit of capacity. Conversely, the policy π^{ra} excels in situations where arrivals are significantly slower compared to the capacity, except in cases where the arrival rate is so slow that all policies yield optimal results. In scenarios of moderate arrival rates, the exponential-index-based policy $\pi^{\mathsf{exp}(\alpha)}$ derived from our worst-case analysis framework outperforms others, even in average-case scenarios.

Explanations of Experimental Results. When $C = 1$, we first observe a sharp decrease in the performance ratio of π^{h} around $\lambda = 3$, which recovers shortly after. The explanation behind this phenomenon is that when $\lambda \ll 1$ the system is empty enough so that the capacity is not binding and all the three algorithms essentially hold orders until deadlines. Their ratios are close to one. When λ becomes around 3 (meaning around 3 arrivals per unit of time), then it becomes possible that π^{h} rejects a fresh order whose height is slightly lower than the currently held (old) order, leading to a sharp decrease on its performance. However, as λ increases, the likelihood of an incoming order having a greater height also rises. Consequently, both π^{h} and OPT start favoring these newer orders, surpassing the likelihood of scenarios where only the optimal offline algorithm makes the switch. This shift leads to a gradual improvement in the performance of π^{h}, albeit in a moderate manner.

A nuanced interpretation can be provided for π^{ra} as well. When the distribution of height follows Absolute Normal in which heights tend to cluster around zero, we can notice that the performance



(a) C=1, Absolute Normal (b) C=1, Uniform (c) C=1, Two-Phase Uniform

(d) C=10, Absolute Normal (e) C=10, Uniform (f) C=10, Two-Phase Uniform

(g) C=20, Absolute Normal (h) C=20, Uniform (i) C=20, Two-Phase Uniform

Figure 4 Average ratios against Optimal Offline (a)-(c) or Skyline (d)-(i) as a function of arrival rate λ.

of π^{ra} continues to deteriorate as λ increases. This decline is rooted in the fact that, unlike π^{h} whose worst-case scenario involves only two orders (see the proof of Proposition 1), π^{ra} grapples with infinitely many arrivals in its worst-case instance (see the proof of Proposition 2). Consequently, the more frequent the occurrence of extended sequences of decreasing orders, the closer we get to this worst-case scenario. By raising λ, we increase the likelihood of detrimental switches under π^{ra} for these long decreasing sequences, ultimately diminishing π^{ra}'s performance. However, when the distribution of heights is not heavily concentrated around zero, such as in the Uniform distribution, the probability of encountering long decreasing sequences becomes much smaller than under the

Absolute Normal distribution. This makes it less likely for π^{ra} to follow a similar path as its worst-case instance. Furthermore, the likelihood of encountering increasing sequences grows, resulting in more instances where π^{ra} and OPT make the same decisions. Taking these two effects into account, under the Uniform or Two-phase Uniform distributions, we observe that the decreasing trend in π^{ra} diminishes, and even reverses, more swiftly compared to the Absolute Normal distribution.

Regarding $\pi^{\mathsf{exp}(\alpha)}$, since it is more balanced than the other two, its performance ratio remains stable when facing two different styles of worst-case instances as described above (one with two arrivals for π^{h} and one with infinitely many arrivals for π^{ra}). The balancing nature implies that the rate of performance change w.r.t. λ for $\pi^{\mathsf{exp}(\alpha)}$ should fall somewhere between that of π^{ra} and π^{h}. This intuition is confirmed in our numerical results, because the increasing and decreasing trends as discussed above seem to offset each other when applied to $\pi^{\mathsf{exp}(\alpha)}$, which ensures that $\pi^{\mathsf{exp}(\alpha)}$ maintains a consistent performance regardless of values in λ.

In regard to the effect of capacity C, one might initially think that scaling both C and λ by the same factor would lead to minimal changes in the average performance ratios. Surprisingly, our experiments show otherwise, as evident from the plots. It appears that this relationship is more additive than multiplicative. This means that increasing both C and λ by the same amount results in roughly unchanged average performance ratios.

Lastly, we observe that a transition in the distribution has a minimal effect. This is evidenced by the high similarity in results for uniform versus two-phase uniform distributions. The results suggest that variations in distribution have only a small influence on the overall outcomes of our experiment.

9. Conclusion

In this paper, we introduce a parsimonious online decision-making model to capture the trade-off between holding a newly-arrived order with a lower chance for consolidation vs. holding an order with a higher chance that has already stayed a while in the pool. We investigate several intuitive online algorithms and establish constant-factor competitive ratio guarantees for their performance. Our methodological contribution is a new framework that introduces the concepts of "parallel partitioning" and "quasi-utility," which allow us to disentangle the actual utility between online algorithms and offline one while simultaneously coupling their quasi-utilities, leading to performance guarantees. Finally, we conduct a concise numerical study to complement our theoretical results, demonstrating that our proposed algorithms achieve a much higher average ratio than the theoretically guaranteed $\mathcal{CR}$ under such stochastic instances. Our model and algorithms are applicable not only to the online retail business but also to the rapidly growing GPU rental market, driven by the recent wave of AI advancements.

Acknowledgments

The authors thank John R. Birge for his insightful comments on the relevance of our model to the GPU rental business, and Barış Ata for drawing our attention to Vast.ai.

References

Abels A, Pitschmann E, Schmand D (2023) Prophet inequalities over time. *Proceedings of the 24th ACM Conference on Economics and Computation*, 1–20, EC '23 (New York, NY, USA: Association for Computing Machinery), ISBN 9798400701047, URL `http://dx.doi.org/10.1145/3580507.3597741`.

Acimovic J, Farias VF (2019) The fulfillment-optimization problem. *Operations Research & Management Science in the Age of Analytics*, 218–237 (INFORMS).

Acimovic J, Graves SC (2015) Making better fulfillment decisions on the fly in an online retail environment. *Manufacturing & Service Operations Management* 17(1):34–51.

Amil A, Makhdoumi A, Wei Y (2022) Multi-item order fulfillment revisited: Lp formulation and prophet inequality, available at SSRN: https://ssrn.com/abstract=4176274.

Amorim P, DeHoratius N, Eng-Larsson F, Martins S (2022) Customer preferences for delivery service attributes in attended home delivery, available at SSRN: https://ssrn.com/abstract=3592597.

Andrews JM, Farias VF, Khojandi AI, Yan CM (2019) Primal-dual algorithms for order fulfillment at urban outfitters, inc. *INFORMS Journal on Applied Analytics* 49(5):355–370.

Ata B, Lee D, Sönmez E (2019) Dynamic volunteer staffing in multicrop gleaning operations. *Operations Research* 67(2):295–314.

AWS (2021) How spot instances work. URL `https://docs.aws.amazon.com/whitepapers/latest/cost-optimization-leveraging-ec2-spot-instances/how-spot-instances-work.html`.

Baek J, Ma W (2022) Bifurcating constraints to improve approximation ratios for network revenue management with reusable resources. *Operations Research* 70(4):2226–2236.

Bai J, Fügener A, Gönsch J, Brunner JO, Blobner M (2021) Managing admission and discharge processes in intensive care units. *Health care management science* 24(4):666–685.

Besbes O, Elmachtoub AN, Sun Y (2019) Static pricing: Universal guarantees for reusable resources. *Proceedings of the 2019 ACM Conference on Economics and Computation*, 393–394.

Buchbinder N, Kimbrel T, Levi R, Makarychev K, Sviridenko M (2013) Online make-to-order joint replenishment model: Primal-dual competitive algorithms. *Operations Research* 61(4):1014–1029.

Canetti R, Irani S (1995) Bounding the power of preemption in randomized scheduling. *Proceedings of the Twenty-Seventh Annual ACM Symposium on Theory of Computing*, 606–615, STOC '95 (New York, NY, USA: Association for Computing Machinery), ISBN 0897917189, URL `http://dx.doi.org/10.1145/225058.225278`.

Chen S, Yan Z, Lim YF (2024) Managing the personalized order-holding problem in online retailing. *Manufacturing & Service Operations Management* 26(1):47–65.

Chrobak M, Epstein L, Noga J, Sgall J, van Stee R, Tichý T, Vakhania N (2003) Preemptive scheduling in overloaded systems. *Journal of Computer and System Sciences* 67(1):183–197, ISSN 0022-0000, URL `http://dx.doi.org/https://doi.org/10.1016/S0022-0000(03)00070-9`.

Chrobak M, Jawor W, Sgall J, Tichý T (2007) Online scheduling of equal-length jobs: Randomization and restarts help. *SIAM Journal on Computing* 36(6):1709–1728, URL `http://dx.doi.org/10.1137/S0097539704446608`.

Corless RM, Gonnet GH, Hare DE, Jeffrey DJ, Knuth DE (1996) On the lambert w function. *Advances in Computational mathematics* 5:329–359.

Dasgupta A, Ghosh A, Nazerzadeh H, Raghavan P (2009) Online story scheduling in web advertising. *Proceedings of the Twentieth Annual ACM-SIAM Symposium on Discrete Algorithms*, 1275–1284, SODA '09 (USA: Society for Industrial and Applied Mathematics).

DeValve L, Wei Y, Wu D, Yuan R (2023) Understanding the value of fulfillment flexibility in an online retailing environment. *Manufacturing & Service Operations Management* 25(2):391–408.

Devanur NR, Jain K, Kleinberg RD (2013) *Randomized Primal-Dual Analysis of RANKING for Online Bipartite Matching*, 101–107 (SIAM), URL `http://dx.doi.org/10.1137/1.9781611973105.7`.

Dickerson JP, Sankararaman KA, Srinivasan A, Xu P (2021) Allocation problems in ride-sharing platforms: Online matching with offline reusable resources. *ACM Transactions on Economics and Computation (TEAC)* 9(3):1–17.

Ekbatani F, Feng Y, Kash I, Niazadeh R (2024) Online job assignment. *SSRN Electronic Journal* URL `https://api.semanticscholar.org/CorpusID:269100314`.

Epstein L, Jeż L, Sgall J, Stee R (2016) Online scheduling of jobs with fixed start times on related machines. *Algorithmica* 74(1):156–176, ISSN 0178-4617, URL `http://dx.doi.org/10.1007/s00453-014-9940-2`.

Feng Y, Caldentey R, Xin L, Zhong Y, Wang B, Hu H (2024) Designing sparse graphs for stochastic matching with an application to middle-mile transportation management. *Management Science* 70(12):8988–9013.

Feng Y, Niazadeh R, Saberi A (2022) Near-optimal bayesian online assortment of reusable resources. *Proceedings of the 23rd ACM Conference on Economics and Computation*, 964–965.

Gao Z, Ling Z, Gupta V, Xin L (2022) Real-time omnichannel fulfillment optimization, available at SSRN: https://ssrn.com/abstract=4314250.

Golrezaei N, Nazerzadeh H, Rusmevichientong P (2014) Real-time optimization of personalized assortments. *Management Science* 60(6):1532–1551.

Golrezaei N, Yao E (2021) Upfront commitment in online resource allocation with patient customers. *arXiv preprint arXiv:2108.03517* .

Gong XY, Goyal V, Iyengar GN, Simchi-Levi D, Udwani R, Wang S (2022) Online assortment optimization with reusable resources. *Management Science* 68(7):4772–4785.

Google (2023) Spot vms. URL `https://cloud.google.com/compute/docs/instances/spot`.

Goyal V, Iyengar G, Udwani R (2020) Online allocation of reusable resources via algorithms guided by fluid approximations. *arXiv preprint arXiv:2010.03983* .

Huo T, Cheung WC (2022) Online reusable resource allocations with multi-class arrivals. *Available at SSRN 4320423* .

Jasin S, Sinha A (2015) An lp-based correlated rounding scheme for multi-item ecommerce order fulfillment. *Operations Research* 63(6):1336–1351.

Kolen AW, Lenstra JK, Papadimitriou CH, Spieksma FC (2007) Interval scheduling: A survey. *Naval Research Logistics (NRL)* 54(5):530–543, URL `http://dx.doi.org/10.1002/nav.20231`.

Lei Y, Jasin S (2020) Real-time dynamic pricing for revenue management with reusable resources, advance reservation, and deterministic service time requirements. *Operations Research* 68(3):676–685.

Ma W (2023) Order-optimal correlated rounding for fulfilling multi-item e-commerce orders. *Manufacturing & Service Operations Management* 25(4):1324–1337.

Manshadi V, Rodilitz S (2020) Online policies for efficient volunteer crowdsourcing. *Proceedings of the 21st ACM Conference on Economics and Computation*, 315–316.

Mehta A, Saberi A, Vazirani U, Vazirani V (2007) Adwords and generalized online matching. *Journal of the ACM (JACM)* 54(5):22–es.

Pauley E (2023) Farewell to the era of cheap ec2 spot instances. URL `https://pauley.me/post/2023/spot-price-trends/`.

Rusmevichientong P, Sumida M, Topaloglu H (2020) Dynamic assortment optimization for reusable products with random usage durations. *Management Science* 66(7):2820–2844.

Simchi-Levi D, Zheng Z, Zhu F (2023) On greedy-like policies in online matching with reusable network resources and decaying rewards. URL `http://dx.doi.org/10.2139/ssrn.3981123`.

Statista (2022) Amazon's escalating logistics costs. URL `https://www.statista.com/chart/17207/amazon-shipping-and-fulfillment-costs/`.

Upwork (2023) Success stories: See how upwork helps businesses like yours grow and succeed. URL `https://www.upwork.com/success-stories`.

Vastai (2024) Vast.ai: Gpu cloud for deep learning. `https://vast.ai/`, accessed: 2024-10-03.

Wagner J, Gabler NB, Ratcliffe SJ, Brown SE, Strom BL, Halpern SD (2013) Outcomes among patients discharged from busy intensive care units. *Annals of Internal Medicine* 159(7):447–455.

Wagner L, Calvo E, Amorim P (2023) Better together! the consumer implications of delivery consolidation. *Manufacturing & Service Operations Management* 25(3):903–920.

Wang Y, Wang T, Wang X, Deng Y, Cao L (2024) Data-driven order fulfillment consolidation for online grocery retailing. *INFORMS Journal on Applied Analytics* 54(3):211–221.

Wei L, Jasin S, Kapuscinski R (2021) Shipping consolidation across two warehouses with delivery deadline and expedited options for e-commerce and omni-channel retailers. *Manufacturing & Service Operations Management* 23(6):1634–1650.

Woeginger GJ (1994) On-line scheduling of jobs with fixed start and end times. *Theor. Comput. Sci.* 130(1):5–16, ISSN 0304-3975, URL `http://dx.doi.org/10.1016/0304-3975(94)90150-3`.

Wu D (2018) Going the extra mile in customer service: How does jd.com make order cancellation so easy?, jD Technology Blog.

Xie Y, Ma W, Xin L (2024) The benefits of delay to online decision-making. *Management Science* Forthcoming.

Xu P, Allgor R, Graves S (2009) Benefits of reevaluating real-time order fulfillment decisions. *Manufacturing & Service Operations Management* 11(2):340–355.

Zhao Y, Wang X, Xin L (2020) Multi-item online order fulfillment in a two-layer network, available at SSRN: https://ssrn.com/abstract=3675117.

This page is intentionally blank. Proper e-companion title page, with INFORMS branding and exact metadata of the main paper, will be produced by the INFORMS office when the issue is being assembled.

E-Companion

Appendix A: Stochastic Reward Processes

Recall that in the main paper, we assume that the reward process is deterministic, with the reward rate $h(t)$ being either time-invariant (core model) or time-varying (extension in Section 7). In this section, we demonstrate that when the reward process becomes stochastic, it can be reformulated as an equivalent deterministic reward process by defining appropriate reward rates, provided it has a deterministic compensator function. The extension to stochastic reward processes is particularly relevant in applications such as order consolidation, where the multiorder cost-saving process for each customer is a jump process, adding a stochastic reward when a new multiorder is placed.

Specifically, suppose that customer i's multiorder process follows a jump process $X_i(t)$. Let $\mathcal{F}_i$ denote the natural filtration associated to the process $\{X_i(t)\}_t$, and $\mathcal{F} = \bigcup_i \mathcal{F}_i$ be the natural filtration associated with the joint process involving all $X_i(t)$. Define $\lambda_i(t)$ as the density of the multiorder arrival process at time t, where $\lambda_i(t) = 0$ for all $t \le t_i$, and let $w_i(t)$ represent the associated savings per multi-order hit at time t, i.i.d. drawn from a random variable W_i. $X_i(t)$ can also be interpreted as the accumulated reward process for customer i by time t, where $X_i(t) = 0$ for all $t \le t_i$, since the initial order is placed at t_i. Define $h_i(t) \triangleq \lambda_i(t)E[W_i]$, where $\int_0^t h_i(s)ds$ represents the compensator for $X_i(t)$, capturing the expected savings up to time t. Thus, $M_i(t) \triangleq X_i(t) - \int_0^t h_i(s)ds$ is a zero-mean martingale w.r.t. $\mathcal{F}_i$. As customers' processes are independent, $M_i(t)$ is also a zero-mean martingale w.r.t. $\mathcal{F}$. The dispatch time $\tau_i \ge t_i$ for order i, determined by a non-anticipative policy (including the clairvoyant optimal online policy, which knows future customers but not their multiorder realizations in advance), is a stopping time w.r.t. $\mathcal{F}$ (though not necessarily for each $\mathcal{F}_i$). Combining all these together, we have multiple martingales, each associated with a stopping time w.r.t. $\mathcal{F}$. By applying the optional stopping theorem to each martingale $M_i(t)$ and its stopping time τ_i, we obtain $E[M_i(\tau_i)] = E[M_i(0)] = 0$, which implies

$$E[X_i(\tau_i)] = E\left[\int_0^{\tau_i} h_i(s)ds\right] = E\left[\int_{t_i}^{\tau_i} h_i(s)ds\right].$$

Here, $E[X_i(\tau_i)]$ represents the expected cost savings accumulated through multiorder consolidation, while $E\left[\int_0^{\tau_i} h_i(s)ds\right]$ denotes the expected accumulated deterministic reward rate $h_i(t)$ up to the stopping time (dispatch time) τ_i under the same policy. Under this martingale property, the expected reward of any online algorithm (including ours) is equivalent to that of operating on the deterministic reward-rate functions $h_i(t)$ for all i.

Note that for problems with a stochastic reward process, we benchmark our policies against a clairvoyant optimal online algorithm. This algorithm knows all customer arrivals (including their timing, lifespan, and multiorder process) in advance but learns the exact multiorder realizations only as they occur (see Golrezaei et al. (2014) for an example of this type of benchmark). It solves the optimal dynamic program based on all arrivals, a computationally challenging task. Importantly, this benchmark is clairvoyant optimal online, not offline, ensuring that OPT's dispatch decisions remain stopping times under the natural filtration. Similarly, the clairvoyant's expected reward matches the optimal offline benchmark in the reduced-form model, where rewards are deterministically accumulated based on $h_i(t)$, as described in Subsection 3.1.

Lastly, we note that this abstraction applies to various processes, including the broad class of compound nonhomogeneous Poisson processes. Here, for any customer, multiorder arrivals follows a nonhomogeneous Poisson process, and the benefit from each realized multiorder is an i.i.d. random variable. In its simplest form, it reduces to a Poisson process with a constant arrival rate and fixed jump size, representing the cost saved per consolidated parcel. In conclusion, a general stochastic reward process in our problem can be reformulated as an equivalent deterministic reward process with reward rates $h_i(t)$ over time, captured by our parsimonious model. This also justifies the constant reward rate assumption in the core model, as discussed in Subsection 3.1.

Appendix B: Proofs of Section 3

B.1. Proofs in Subsection 3.4

B.1.1. Proof of Lemma 1

Proof of Lemma 1. To prove this, we first provide an adjustment to the algorithm π^{h}, namely $\widetilde{\pi^{\mathsf{h}}}$. The dispatching decisions in $\widetilde{\pi^{\mathsf{h}}}$ are exactly same as in π^{h}, except for orders that π^{h} keeps until their deadline, i.e., for a whole duration of 1. For such orders, we assume that $\widetilde{\pi^{\mathsf{h}}}$ keeps them twice as much, i.e., for a total duration of 2. Notice that neither the capacity constraint nor the deadline constraint needs to be satisfied for $\widetilde{\pi^{\mathsf{h}}}$, as this policy is only for the sake of analysis and will not be used for implementation. Lastly, we will prove the following two inequalities:

1. $2\, U_{\pi^{\mathsf{h}}}(\mathcal{I}) \geq U_{\widetilde{\pi^{\mathsf{h}}}}(\mathcal{I})$:

The proof of this inequality simply comes from the definition of $U_{\widetilde{\pi^{\mathsf{h}}}}(\mathcal{I})$, as all orders that are send out before their deadline under π^{h} will be held for the same duration under $\widetilde{\pi^{\mathsf{h}}}$ as well, and all those which are kept until their deadline, i.e., for duration 1, will now be kept for duration 2 under $\widetilde{\pi^{\mathsf{h}}}$. Therefore, for each order, $\widetilde{\pi^{\mathsf{h}}}$ will get at most twice accumulated reward as π^{h}, now if we sum over all orders we will get the inequality we wanted.

2. $U_{\widetilde{\pi^{\mathsf{h}}}}(\mathcal{I}) \geq \mathsf{Skyline}(\mathcal{I})$:

We claim that at any time t, $\widetilde{\pi^{\mathsf{h}}}$ is holding orders with rewards that dominates that of $\mathsf{Skyline}(\mathcal{I})$. More specifically, at any time t, let $h_{s_1} \geq h_{s_2} \geq ... \geq h_{s_k}, (k \leq C)$, be the distinct orders that $\mathsf{Skyline}(\mathcal{I})$ is keeping. Then we claim that there are distinct orders $h_{i_1} \geq h_{i_2} \geq ... \geq h_{i_k}$ that $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ is holding which satisfies $\forall 1 \leq j \leq k,\ h_{i_j} \geq h_{s_j}$.

First notice that if $\pi^{\mathsf{h}}(\mathcal{I})$ is holding some order at time t, $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ do so as well. Now among the sequence $h_{s_1} \geq h_{s_2} \geq ... \geq h_{s_k}$ consider the order with highest reward that $\pi^{\mathsf{h}}(\mathcal{I})$ is not holding at time t, say s_j. This means that $\pi^{\mathsf{h}}(\mathcal{I})$ must have sent out order s_j at some time $t' \leq t$. This implies that, at the time t', $\pi^{\mathsf{h}}(\mathcal{I})$ was keeping C orders all with rewards at least h_{s_j}. Now, look at the time interval $[t', t]$. First, we know that, unless hitting the deadline, $\pi^{\mathsf{h}}(\mathcal{I})$ will only replace an order with another one with higher reward rate. Second, if an order hit its deadline while $\pi^{\mathsf{h}}(\mathcal{I})$ was keeping it, then $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ will keep it for a whole extra unit of time. Therefore, any of such higher reward orders that $\pi^{\mathsf{h}}(\mathcal{I})$ is keeping at time t' will either remain to be hold until t, replace with some higher reward (could be multiple replaces over the time interval until t, but each of them has higher reward rate with the previous one in this "replacement chain"), or hit the deadline. Notice that in all of these three cases, as $t - t' < 1$, $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ will hold at least C number of orders at time t

which weakly dominates (in terms of their reward rate) those ones that $\pi^{\mathsf{h}}(\mathcal{I})$ was holding at time t', which themselves weakly dominated the s_j that $\mathsf{Skyline}(\mathcal{I})$ is holding at time t.

Finally, for $s_1, ..., s_{j-1}$ we knew that $\pi^{\mathsf{h}}(\mathcal{I})$ and so as $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ are holding them until at least time t (by definition of s_j). Combining this with the statement that we showed earlier - i.e., $\widetilde{\pi^{\mathsf{h}}}(\mathcal{I})$ is keeping at least C orders with rewards at least h_{s_j} (which is greater or equal than the reward rate of the rest of the orders s_{j+1} to s_k) - we can conclude the proof of this dominance claim (and so as the second inequality) by setting $i_l = s_l$, $\forall 1 \leq l \leq j-1$, and then choosing $k-(j-1)$ highest reward orders from the rest of the orders that $\widetilde{\pi^{\mathsf{h}}}$ is holding at time t - each of which had a reward rate of at least h_{s_j}.

□

B.1.2. Proof of Proposition 1

Proof of Proposition 1. The proof of $\mathcal{CR}(\pi^{\mathsf{h}}) \geq \frac{1}{2}$ is readily attained from Lemma 1. The rest is to show that this ratio is tight, by providing a simple bad instance. Instance $\mathcal{I}$ consists of only 2 types of orders, where each type has C identical orders. The first type is $z_1 := (t_1 = 0, h_1 = 1)$, and the second type is $z_2 := (t_2 = 1^-, h_2 = 1^-)$. Under this instance $\mathcal{I}$, π^{h} will only hold all the first types till their deadline ($t = 1$) and dispatches all second types immediately, whereas the optimal offline $\mathsf{OPT}(\mathcal{I})$ will dispatch all the first types when second types arrive and hold them till their deadline ($t = 2^-$). It is clear to see that $U^{\mathsf{h}}_{\pi}(\mathcal{I}) = C$ and $\mathsf{OPT}(\mathcal{I}) = C^- + C^-$, which results in:

$$\mathcal{CR}(\pi^{\mathsf{h}}) \leq \frac{U^{\mathsf{h}}_{\pi}(\mathcal{I})}{\mathsf{OPT}(\mathcal{I})} = \frac{1}{2},$$

concluding the tightness of the analysis, i.e., $\mathcal{CR}(\pi^{\mathsf{h}}) = \frac{1}{2}$. □

B.2. Proofs in Subsection 3.5

We will prove Proposition 2 in this subsection.

First, we will provide the definitions and notations that we use to analyse the competitive ratio of the policy π^{ra}. It is true that the algorithm has no non-trivial information about how the optimal offline policy is making its decisions; nonetheless, we can use the decisions of the optimal offline in order to provide fitting structure for our competitive ratio analysis of π^{ra} against the unknown-to-algorithm optimal offline.

In the following we will construct a continuum of bijective functions, $\forall t \in \mathbb{R}$, $\mu_t : \mathcal{H}_{\mathsf{OPT}}(t) \to \mathcal{H}_{\pi^{\mathsf{ra}}}(t)$, which are one-to-one correspondences that match each order that is being held by OPT at time t, with a distinct order that is being held by π^{ra} at the same time t. The main purpose of this one-to-one correspondence is that it provides us with a clear way of accounting the rewards for the orders that π^{ra} is holding and then comparing them with the rewards for the orders that OPT is holding them, without being prone to double counting mistakes. We also denote the inverse of this bijective matching at each time t by $\mu_t^- : \mathcal{H}_{\pi^{\mathsf{ra}}}(t) \to \mathcal{H}_{\mathsf{OPT}}(t)$.

Now let us show how we will construct this one-to-one correspondence at each time. Notice that whenever a new order arrives, as long as there is some capacity available, both of OPT and π^{ra} will start holding that. Additionally, both of these policies face the same instance, meaning that both of them will see the same arrival time and lifespan for each order. This will tell us that since time $t = 0$, as long as the capacity constraint is not binding, both of them will hold the exact same set of orders at each time. Let us denote

t_0 as the first time that the capacity constraint will become binding. With this in mind, we set $\mu_t(i) = i \in \mathcal{H}_{\pi^{\mathsf{ra}}}(t), \forall t \in [0, t_0), i \in \mathcal{H}_{\mathsf{OPT}}(t)$.

In the time intervals that there is no change to either of $\mathcal{H}_{\pi^{\mathsf{ra}}}(t)$ and $\mathcal{H}_{\mathsf{OPT}}(t)$, we also do not change the matching $\mu_t(.)$. Also, notice that whenever an order hits its deadline either of the two policies has to dispatch it, but we can think of it as they are replacing it with an order with zero reward. Moreover, the index (remaining area) of such an order at the time of its deadline is also zero. Therefore, as for the π^{ra}, we can even view this dispatch as a deliberate action to an order with same zero index (i.e., to a zero reward order) rather than viewing it such that the algorithm is forced to dispatch that order whose lifespan is finished. With this in mind, the remaining part for us to show in order to fully characterize $\mu_t(.)$ at all times is to show how $\mu_t(.)$ will change at times when either π^{ra} or OPT are accepting a new arrival.

Suppose order i arrives to the system. There are 4 possible cases that can happen, and we will explicitly state how the matching should adapt in each of them.

1. Neither OPT nor π^{ra} accepts order i: In this case, μ_{t_i} continues to be what it was before t_i without any change.

2. Only OPT accepts i: In this case, consider the order which would be replaced with i by OPT at time t_i. Let us call that order j. The matching at this time, μ_{t_i}, will change in the following manner: we connect i to what j was matched to, i.e., $\mu_{t_i}(i) \leftarrow \mu_{t_i^-}(j)$, and we do not change anything else.

3. Only π^{ra} accepts i: In this case, we do a similar change as in the previous case, but now on the π^{ra} side. More specifically, let j to be the order that is being replaced with i by π^{ra} at time t_i. Also let k to be the match of j right before time t_i, i.e., $k = \mu^-_{t_i^-}(j)$. Then, we will change the match of k at t_i from j to be i, $\mu_{t_i}(k) \leftarrow i$. Again, everything else will remain unchanged.

4. Both π^{ra} and OPT accepts i: In this case, let j_π and j_{OPT} be the orders that are being replaced with i at time t_i by π^{ra} and OPT, respectively. There are two possible scenarios that might have happened:

   (a) $\mu_{t_i^-}(j_{\mathsf{OPT}}) = j_\pi$: In this scenario, both of these orders will be discarded by the corresponding policies, and both policies will accept i. As a result the only change we need to do is to set $\mu_{t_i}(i) = i$.

   (b) $\mu_{t_i^-}(j_{\mathsf{OPT}}) \neq j_\pi$: In this scenario, not only do we need to set $\mu_{t_i}(i) = i$, but also we have to match together the matches of both j_{OPT} and j_π, i.e., $\mu_{t_i}(k) \leftarrow \mu_{t_i^-}(j_{\mathsf{OPT}})$ where $k = \mu^-_{t_i^-}(j_\pi)$.

Notice that we also discard the orders from domain of the matching for orders that get dispatched since their dispatch time.

Remark EC.1. Note that from the way we construct matching μ_t, we can easily deduce that it would be, indeed, a one-to-one correspondence from $\mathcal{H}_{\mathsf{OPT}}(t)$ to $\mathcal{H}^{\mathsf{ra}}_\pi(t)$.

Now that we have built our matching μ_t over time, let us also define another function that determines for any order i what is the sequence of its matched orders (on the π^{ra}'s side) over the time interval it was being held by OPT. More formally, for any order i we define $\sigma_i : [t_i, d_i^*) \to [N]$, where d_i^* was the dispatch time of i by OPT. For any order i, we will set:

$$\sigma_i(t) \triangleq \mu_t(i), \quad \forall t \in [t_i, d_i^*).$$

With this way of matching the orders that OPT is keeping with the ones that π^{ra} is keeping, we will have the following properties hold:

Claim EC.1. *For any order $i \in [N]$ such that $t_i < d_i^*$, the following two properties hold for the orders in the sequence $\sigma_i(.)$:*

1. *At time t_i, the remaining area of the order $\sigma_i(t_i)$ is at least as large as that of order i:*

$$h_{\sigma_i(t_i)}(t_{\sigma_i(t_i)} + 1 - t_i) \geq h_i.$$

2. *At any time $t \in (t_i, d_i^*)$, the remaining area of the order $\sigma_i(t)$ at the time t is at least as large as that of the order $\sigma_i(t^-)$:*

$$h_{\sigma_i(t)}(t_{\sigma_i(t)} + 1 - t) \geq h_{\sigma_i(t^-)}(t_{\sigma_i(t^-)} + 1 - t).$$

Proof of Claim EC.1. The proof follows from the way we have constructed the matching, as well as the fact that the algorithm always keeps the orders with the highest remaining area. Now let us formally state the proof for each part of the claim:

1. Order i arrives at time t_i, now there are two cases:

(a) if algorithm also accepts to hold i (case 4.a): in this case by definition we have $\sigma_i(t_i) = i$ and the statement of this part hold clearly.

(b) if algorithm never hold order i (immediate dispatch of i by algorithm, case 2): in this case, we did $\sigma_i(t_i) = \mu_{t_i}(i) \leftarrow \mu_{t_i^-}(j)$. From the fact that order $\mu_{t_i^-}(j)$ is still kept in the system and order i was dispatched by π^{ra}, we get that its remaining area at time t_i should have been higher, which is exactly what we wanted to show.

2. To show this part, if $\sigma_i(t^-) = \sigma_i(t)$, then clearly the expression holds with equality. On the other hand, if $\sigma_i(t^-) \neq \sigma_i(t)$, it means that either case 3 or case 4.b) has happened, and in both of them π^{ra} dispatches $\sigma_i(t^-)$ instead of $\sigma_i(t)$. Thus, again by the fact that π^{ra} keeps the orders with higher remaining area at t, we immediately get: $h_{\sigma_i(t)}(t_{\sigma_i(t)} + 1 - t) \geq h_{\sigma_i(t^-)}(t_{\sigma_i(t^-)} + 1 - t)$.

□

The next lemma will provide a comparison between the total reward that OPT is acquiring from any order i and the reward that π^{ra} is accumulating over that period of time from the orders that are matched to i, namely $\sigma_i(.)$.

Lemma EC.1. *For any order $i \in [N]$ such that $t_i < d_i^*$, the following inequality holds:*

$$\int_{t_i}^{d_i^*} h_{\sigma_i(t)} \; dt \geq (1 - \frac{1}{e}) \times h_i(d_i^* - t_i).$$

Proof of Lemma EC.1. To prove this lemma, we make use of Claim EC.1, but before that let us define some simplifying notation. In the sequence of orders $\sigma_i(.)$, let us define τ_j as the j-th changing time in the sequence, with $\tau_0 \triangleq t_i$ and $\tau_{n_i} \triangleq d_i^*$, where n_i is the total number of matches to order i. In other words, τ_j is the j-th time in the sequence that $\sigma_i(\tau_j) \neq \sigma_i(\tau_j^-)$. Let us also define $\forall 1 \leq j \leq n_i : \alpha_j \triangleq \tau_j - \tau_{j-1}, \beta_j \triangleq (1 + t_{\sigma_i(\tau_{j-1})} - \tau_{j-1})$, where α_j would be the length of time that i is matched to its j-th match and β_j would be the remaining lifespan of i's j-th match at the time when its matched interval starts (at time τ_{j-1}). Additionally, for conciseness we refer to $h_{\sigma_i(\tau_{j-1})}$ as h_{i_j}. With these in mind, from Claim EC.1 we have:

$$h_{i_1}\beta_1 \geq h_i,$$

$$h_{i_{j+1}}\beta_{j+1} \geq h_{i_j}(\beta_j - \alpha_j) = h_{i_j}\beta_j(1 - \frac{\alpha_j}{\beta_j}) \quad \forall 1 \leq j \leq n_i - 1,$$

where the first inequality is equivalent to the first part of the claim, and the second inequality will be derived by setting $t=\tau_j$ for all $1\le j\le n_i-1$, in the second part of the claim. By using the second inequality in a cascade manner, we can get:

$$\forall 1\le j\le n_i-1:\ h_{i_{j+1}}\beta_{j+1}\ge h_{i_1}\beta_1\prod_{k=1}^{j}(1-\frac{\alpha_k}{\beta_k})\ \Rightarrow\ h_{i_{j+1}}\alpha_{j+1}\ge h_{i_1}\beta_1\left(\prod_{k=1}^{j}(1-\frac{\alpha_k}{\beta_k})\right)\frac{\alpha_{j+1}}{\beta_{j+1}}.$$

Then, if we sum up the above inequality for all j from 0 to n_i-1 we get:

$$\begin{aligned}\int_{t_i}^{d_i^*} h_{\sigma_i(t)}\ dt &= \sum_{j=0}^{n_i-1} h_{i_{j+1}}\alpha_{j+1}\ge h_{i_1}\beta_1\sum_{j=0}^{n_i-1}\left(\left(\prod_{k=1}^{j}(1-\frac{\alpha_k}{\beta_k})\right)\frac{\alpha_{j+1}}{\beta_{j+1}}\right)\\ &= h_{i_1}\beta_1\left(1-\prod_{j=1}^{n_i}(1-\frac{\alpha_j}{\beta_j})\right)\ge h_{i_1}\beta_1\left(1-\prod_{j=1}^{n_i}(1-\alpha_j)\right),\end{aligned}$$

where the second inequality uses the fact that $\alpha_j\le\beta_j\le 1$, which is true by definition as $t_{\sigma_i(\tau_{j-1})}\le\tau_{j-1}\le\tau_j\le t_{\sigma_i(\tau_{j-1})}+1$.

Moreover, notice that we have $\sum_{j=1}^{n_i}\alpha_j=d_i^*-t_i$, so we can further lower bound the last inequality by considering the maximum possible value for the product expression, which will happen when all of its terms are equal to each other, i.e., $\forall j:\alpha_j=\frac{d_i^*-t_i}{n_i}$. Lastly, by using inequalities $h_{i_1}\beta_1\ge h_i$ and $\forall x: e^x\ge 1+x$, we will get:

$$\int_{t_i}^{d_i^*} h_{\sigma_i(t)}\ dt\ge h_{i_1}\beta_1\left(1-\prod_{j=1}^{n_i}(1-\alpha_j)\right)\ge h_{i_1}\beta_1\left(1-(1-\frac{d_i^*-t_i}{n_i})^{n_i}\right)\ge h_i\left(1-e^{-(d_i^*-t_i)}\right).$$

Finally, the function $\frac{1-e^{-x}}{x}$ is a decreasing function, thereby finishing the proof (as $d_i^*-t_i\le 1$):

$$\int_{t_i}^{d_i^*} h_{\sigma_i(t)}\ dt\ge h_i\left(1-e^{-(d_i^*-t_i)}\right)\ge h_i(d_i^*-t_i)(1-\frac{1}{e}).$$

□

We are now ready to finish the proof of Proposition 2, in which we prove the lower bound of $1-\frac{1}{e}$ as well as provide the tight upper bound (bad instance) for it.

Proof of Proposition 2. Due to the fact that μ_t was a one-to-one correspondence at all time, we are sure that at any point in time t, we do not double count any order that π^{ra} is keeping for different i in $\sigma_i(t)$. As a result, we can write the following:

$$\forall\mathcal{I}:\quad U_\pi^{\mathsf{ra}}(\mathcal{I})=\sum_{\{i\in[N]|t_i<d_i^*\}}\left(\int_{t_i}^{d_i^*} h_{\sigma_i(t)}\ dt\right).$$

Remember that $\mathsf{OPT}(\mathcal{I})=\sum_{i\in[N]}h_i(d_i^*-t_i)=\sum_{\{i\in[N]|t_i<d_i^*\}}h_i(d_i^*-t_i)$, so in order to get any competitive ratio X for π^{ra}, it is sufficient to show that $\forall i\in[N]$ s.t. $t_i<d_i^*$:

$$\frac{\int_{t_i}^{d_i^*} h_{\sigma_i(t)}\ dt}{h_i(d_i^*-t_i)}\ge X.$$

From Lemma EC.1 we know that the inequality holds when $X=1-\frac{1}{e}$, thereby concluding that $\mathcal{CR}(\pi^{\mathsf{ra}})\ge 1-\frac{1}{e}$.

Finally, to see that the competitive ratio of π^{ra} is tight, i.e., $\mathcal{CR}(\pi^{\mathsf{ra}})=1-\frac{1}{e}$, we will provide the following worst-case instance. The instance $\mathcal{I}$ consists of a repeated sequence of continuum arrival of orders. More

specifically, for any round k corresponding to the time interval $[k, k+1)$, we have a continuum of arrivals in the following manner: at time $k+t \in [k, k+1), \forall t \in [0,1)$, we input an order with height e^{-t}, i.e., $z_{t+k} = (t+k, e^{-t})$. Because the gradient of the height function (e^{-t}) for any $t \in (0,1)$ is equal to negative of the height of the order z_{t+k} for any round k, which itself is equal to the gradient of the index function of order z_{t+k}, we can conclude that the policy π^{ra} will accept the order z_{t+k+dt}. As for the orders z_k, i.e., when $t=0$, we know that the algorithm will accept it because $h_{z_k} = 1$, whereas $h_{z_{k^-}} = \frac{1}{e}$ which is the order that π^{h} is holding at the end of the previous round. On the other hand, the optimal offline will only hold orders z_k, and dispatches all the rest of orders z_{k+t}, $\forall k \in \mathbb{Z}_+, t \in (0,1)$ immediately as they arrive. Note that if $C > 1$, we can just duplicate each order C times, and all the above argument will still hold. This way $\mathsf{OPT}(\mathcal{I})$ is always holding C orders with reward 1 at each time t (only all the integer orders), while $\pi^{\mathsf{ra}}(\mathcal{I})$ is accepting to hold all orders for infinitesimal amount of time. Thus, the total reward in each interval $[k, k+1), \forall k$ of $\mathsf{OPT}(\mathcal{I})$ would be C, and that of $\pi^{\mathsf{ra}}(\mathcal{I})$ would be equal to $C\int_0^1 e^{-t}dt = C(1-\frac{1}{e})$, which proves the tightness of $1-\frac{1}{e}$ for the competitive ratio, i.e., $\mathcal{CR}(\pi^{\mathsf{ra}}) = 1-\frac{1}{e}$.

□

Appendix C: Proofs of Section 4

C.1. Proofs in Subsection 4.2

For simplicity, we slightly abuse the notation and refer to $\widetilde{\pi^{\mathcal{B}}}$ as just $\tilde{\pi}$, in the rest of this subsection.

C.1.1. Proof of Lemma 2

Proof of Lemma 2. First notice that we have: $U_{\pi^{\mathcal{B}}}(\mathcal{I}) = \sum_{i\in[N]} h_i(d_i - t_i)$, $V_{\tilde{\pi},\mathcal{B}}(\mathcal{I}) := \sum_{i\in[N]} \left(\int_{t_i}^{\tilde{d}_i} \mathcal{B}_{r_{z_i}}(t)\ dt\right)$. Hence, it is sufficient to show that for all $i \in [N]$, we have

$$h_i(d_i - t_i) \geq \gamma_a \left(\int_{t_i}^{\tilde{d}_i} \mathcal{B}_{r_{z_i}}(t)\ dt\right). \tag{EC.1}$$

Now let us consider two cases:

1. $d_i < t_i + 1$: In this case, we have $\tilde{d}_i = d_i$. From the definition of γ_a in (6), if we set $z = z_i$ and $t' = d_i \in [t_i, t_i+1)$, then it follows immediately from the first term in *min* of γ_a:

$$\frac{h_i(d_i - t_i)}{\int_{t_i}^{d_i} \mathcal{B}_{r_{z_i}}(t)\ dt} \geq \gamma_a,$$

which implies (EC.1).

2. $d_i = t_i + 1$: In this case, we have $\tilde{d}_i = d_i + 1 = t_i + 2$. Similarly, from the definition of γ_a in (6), if we set $z = z_i$, then it follows immediately from the second term in *min* of γ_a:

$$\frac{h_i}{\int_{t_i}^{t_i+2} \mathcal{B}_{r_{z_i}}(t)\ dt} \geq \gamma_a,$$

which implies (EC.1).

The proof is completed. □

C.1.2. Proof of Lemma 3

Before going into the proof, let us first state the following remark.

Remark EC.2. We can assume w.l.o.g. that the system is always full under $\pi^{\mathcal{B}}$, as we can add at any time any arbitrary number of dummy orders whose heights are all equal to 0.

Proof of Lemma 3. We can rewrite the LHS and RHS of (8) in the following manner:

- $V_{\tilde{\pi},\mathcal{B}}(\mathcal{I}) := \sum_{i\in[N]} \left(\int_{t_i}^{\tilde{d}_i} \mathcal{B}_{r_{z_i}}(t)\ dt\right) = \int_0^{T+1} \left(\sum_{i\in\mathcal{H}_{\tilde{\pi}}(t)} \mathcal{B}_{r_{z_i}}(t)\right)\ dt \geq \int_0^T \left(\sum_{i\in\mathcal{H}_{\tilde{\pi}}(t)} \mathcal{B}_{r_{z_i}}(t)\right) dt;$
- $V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}) := \sum_{i\in[N]} \left(\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt\right) = \int_0^T \left(\sum_{i\in\mathcal{H}_{\mathsf{OPT}}(t)} \mathcal{B}_{r_{z_i}}(t)\right)\ dt.$

Thus, it is sufficient to show that for all $t \in [0,T)$, we have

$$\sum_{i\in\mathcal{H}_{\tilde{\pi}}(t)} \mathcal{B}_{r_{z_i}}(t) \geq \sum_{i\in\mathcal{H}_{\mathsf{OPT}}(t)} \mathcal{B}_{r_{z_i}}(t). \tag{EC.2}$$

Claim EC.2. *Given any time t and any order z_k, if we have*

$$\min_{i\in\mathcal{H}_{\pi^{\mathcal{B}}}(t)} \left\{\mathcal{B}_{r_{z_i}}(t)\right\} \geq \mathcal{B}_{r_{z_k}}(t), \tag{EC.3}$$

then we also have

$$\forall t' \in [t,t+1): \min_{i\in\arg\max^C \left\{\mathcal{B}_{r_{z_j}} \middle| j\in\mathcal{H}_{\tilde{\pi}}(t')\right\}} \left\{\mathcal{B}_{r_{z_i}}(t')\right\} \geq \mathcal{B}_{r_{z_k}}(t').$$

Proof of Claim EC.2. The following fact comes from (5):

$$\exists t \text{ such that } \mathcal{B}_{r_{z_i}}(t) \geq \mathcal{B}_{r_{z_k}}(t) \iff r_{z_i} \geq r_{z_k} \iff \mathcal{B}_{r_{z_i}}(t) \geq \mathcal{B}_{r_{z_k}}(t)\ \forall t.$$

Combining the above with the assumption in (EC.3), we get:

$$\min_{i\in\mathcal{H}_{\pi^{\mathcal{B}}}(t)} \{r_{z_i}\} \geq r_{z_k}.$$

Now notice that from the behavior of the index-based policy $\pi^{\mathcal{B}}$ we know that, unless any of its orders hit the end of its lifespan and the policy is forced to dispatch that order, $\pi^{\mathcal{B}}$ dispatches an order k at some time t only if there is a newly-arrived order i whose index at the time is higher than that of order k's, i.e., $f_{z_i}(t) \geq f_{z_k}(t)$ - which itself is equivalent to $\mathcal{B}_{r_{z_i}}(t) \geq \mathcal{B}_{r_{z_k}}(t)$, or simply $r_{z_i} \geq r_{z_k}$.

Now let us prove the claim. Fix an arbitrary t and $t' \in [t,t+1)$, define $D[t,t']$ as the set of the orders that $\pi^{\mathcal{B}}$ dispatches in the time interval $[t,t']$ due to the reason they have hit the end of their lifespans. From the way we defined $\tilde{\pi}$ we know that it will hold all these orders for an extra unit of time after their deadline. From this we can deduce that because $t' \in [t,t+1)$, then all of $D[t,t']$ would be still being held by $\tilde{\pi}$ at time t', i.e., $D[t,t'] \subseteq \mathcal{H}_{\tilde{\pi}}(t')$.

Let us first define relation $\preceq$ between two sets A and B. We say $A \preceq B$, if $|A| \leq |B|$ and if we rank the orders in each of them based on the orders' r_z, then $\forall l \in [|A|]$, the $l-th$ highest r_z in A is no greater than the $l-th$ highest r_z in B.

Now consider $\mathcal{H}_{\pi^{\mathcal{B}}}(t)$. This set consists of C number of orders. Until time t' each of those orders might experience 3 types of fate: i) remain in the system, ii) might get replace multiple times (without ever - the original order or any of its replacement chain - hitting their deadline), and iii) might hit their deadline after some (possibly 0) number of replacements (and probably get replaced by a zero-reward order). Let us name the set of orders in each of these fate classes as F_1, F_2, F_3, respectively. First, by definition of $\pi^{\mathcal{B}}$, we have $\forall s: \mathcal{H}_{\pi^{\mathcal{B}}}(s) \subseteq \mathcal{H}_{\tilde{\pi}}(s)$. Therefore, we immediately get: $F_1 \subseteq \mathcal{H}_{\pi^{\mathcal{B}}}(t') \subseteq \mathcal{H}_{\tilde{\pi}}(t')$. Second, we get: $F_3 \preceq D[t,t'] \subseteq \mathcal{H}_{\tilde{\pi}}(t') \backslash \mathcal{H}_{\pi^{\mathcal{B}}}(t')$, because of the fact that, in each of the replacement chains, the r_z sequence is increasing, and the final element of the chain is indeed in $D[t,t']$ by definition. Lastly, as for the second type,

F_2, again each of the chains will have an increasing sequence of r_z, and for this type, the chain is still alive by time t', meaning the elements of these chains at time t' is also an element of $\mathcal{H}_{\tilde{\pi}}(t')\backslash\{\mathcal{H}_{\pi^{\mathcal{B}}}(t')\cup D[t,t']\}$. Therefore, we get $F_2 \preceq \mathcal{H}_{\tilde{\pi}}(t')\backslash\{\mathcal{H}_{\pi^{\mathcal{B}}}(t')\cup D[t,t']\}$.

Combining all these three pieces together, we can get: $\mathcal{H}_{\pi^{\mathcal{B}}}(t) \preceq \mathcal{H}_{\tilde{\pi}}(t')$, and because size of $\mathcal{H}_{\pi^{\mathcal{B}}}(t)$ was C, then together with that "equivalent" assumption of the claim we can get:

$$r_{z_k} \leq \min_{i\in\mathcal{H}_{\pi^{\mathcal{B}}}(t)}\{r_{z_i}\} \leq \min_{i\in\arg\max^C\left\{\mathcal{B}_{r_{z_j}}\,\middle|\,j\in\mathcal{H}_{\tilde{\pi}}(t')\right\}}\{r_{z_i}\},$$

which implies

$$\min_{i\in\arg\max^C\left\{\mathcal{B}_{r_{z_j}}\,\middle|\,j\in\mathcal{H}_{\tilde{\pi}}(t')\right\}}\{\mathcal{B}_{r_{z_i}}(t')\} \geq \mathcal{B}_{r_{z_k}}(t').$$

The proof of the claim is completed. □

Now let us prove equation (EC.2) with the help of Claim EC.2 for any arbitrary time t.

We divide $\mathcal{H}_{\mathsf{OPT}}(t)$ into two subset, $\mathcal{J} \triangleq \mathcal{H}_{\mathsf{OPT}}(t)\cap\mathcal{H}_{\pi^{\mathcal{B}}}(t)$ and $\mathcal{D} \triangleq \mathcal{H}_{\mathsf{OPT}}(t)\setminus\mathcal{H}_{\pi^{\mathcal{B}}}(t)$. Now notice for any $k\in\mathcal{D}$, we know that the policy $\pi^{\mathcal{B}}$ must have dispatched that order sometime along the way before time t, i.e., $d_k < t$. From the fact that order k is being kept by OPT until time t we conclude that order k had not reached its deadline at time d_k, meaning that the $\pi^{\mathcal{B}}$ had intentionally dispatched order k at time d_k. This means that the indices of all orders that $\pi^{\mathcal{B}}$ was going to hold at time d_k must have been all at least as high as the index of order k at that time d_k; in other words,

$$\min_{i\in\mathcal{H}_{\pi^{\mathcal{B}}}(d_k)}\{\mathcal{B}_{r_{z_i}}(d_k)\} \geq \mathcal{B}_{r_{z_k}}(d_k).$$

Next we will use the claim, where you replace t, t' in the claim by d_k, t respectively. Now from the fact that the order k must have been arrived as late as its dispatch time by $\pi^{\mathcal{B}}$ and its deadline must have been as early as its dispatch time by OPT, i.e., $t_k \leq d_k \leq t \leq t+1$, we conclude that $t\in[d_k, d_k+1)$. Combining the above with (C.1.2), we see that the assumption in the claim holds. Therefore,

$$\min_{i\in\arg\max^C\left\{\mathcal{B}_{r_{z_j}}\,\middle|\,j\in\mathcal{H}_{\tilde{\pi}}(t)\right\}}\{\mathcal{B}_{r_{z_i}}(t)\} \geq \mathcal{B}_{r_{z_k}}(t).$$

Finally, notice that we have showed the above inequality $\forall k\in\mathcal{D}$. Moreover, notice that $|\mathcal{H}_{\mathsf{OPT}}(t)| \leq C$ as well as $\mathcal{J}\subseteq\mathcal{H}_{\pi^{\mathcal{B}}}(t)\subseteq\mathcal{H}_{\tilde{\pi}}(t)$. It follows that

$$\mathcal{H}_{\mathsf{OPT}}(t) \preceq \mathcal{H}_{\tilde{\pi}}(t) \implies \sum_{i\in\mathcal{H}_{\tilde{\pi}}(t)}\mathcal{B}_{r_{z_i}}(t) \geq \sum_{i\in\mathcal{H}_{\mathsf{OPT}}(t)}\mathcal{B}_{r_{z_i}}(t).$$

The proof of Lemma 3 is completed.

□

C.1.3. Proof of Lemma 4

Proof of Lemma 4. The argument is similar to the one of Lemma 2. First, notice that we have: $\mathsf{OPT}(\mathcal{I}) = \sum_{i\in[N]} h_i(d_i^* - t_i)$, $V_{\mathsf{OPT},\mathcal{B}}(\mathcal{I}) := \sum_{i\in[N]} \left(\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt\right)$. Hence, it is sufficient to show that for all $i \in [N]$, we have

$$\left(\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt\right) \geq \gamma_o h_i(d_i^* - t_i).$$

This inequality itself is again a direct consequence of the definition of γ_o. If we set $z = z_i$ and $t' = d_i^* \in [t_i, t_i + 1)$, then it follows immediately from the definition of γ_o in (7):

$$\frac{\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt}{h_i(d_i^* - t_i)} \geq \gamma_o,$$

completing the proof.

□

C.1.4. Proof of Theorem 1

Proof of Theorem 1. To prove Theorem 1, we combine the result of all three Lemmas 2 to 4 together, and we can get what we want:

$$U_{\pi^{\mathcal{B}}}(\mathcal{I}) \geq \gamma_a \sum_{i\in[N]} \left(\int_{t_i}^{\tilde{d}_i} \mathcal{B}_{r_{z_i}}(t)\ dt\right) \geq \gamma_a \sum_{i\in[N]} \left(\int_{t_i}^{d_i^*} \mathcal{B}_{r_{z_i}}(t)\ dt\right) \geq \gamma_a \gamma_o \mathsf{OPT}(\mathcal{I}).$$

□

C.2. Proofs in Subsection 4.3

C.2.1. Proof of Lemma 5

Proof of Lemma 5. Please refer to Section 7 and Lemma 7 for the derivation of γ_a and γ_o under a more generalized setting, in which the reward rate of each order can vary over time. This setting is an special case of that when $R = 1$ (refer to there for definition of $\mathfrak{r}(.)$ and R), and the height of each order is a constant function (meaning $\mathfrak{r}(.) = 1$) which clearly satisfies both homogeneity and non-increasing assumptions needed in Lemma 7. □

C.2.2. Proof of Theorem 2

Proof of Theorem 2. The lower bound part, i.e., $\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)}) \geq \frac{1-e^{-\alpha^*}}{\alpha^*} \geq 0.68$ can be readily achieved by combining Theorem 1 and Lemma 5. The rest is to show that this ratio is, indeed, tight. We prove this by providing a bad instance $\mathcal{I}$ under which the performance ratio of $\pi^{\mathsf{exp}(\alpha^*)}(\mathcal{I})$ against $\mathsf{OPT}(\mathcal{I})$ is, in fact, equal to $\frac{1-e^{-\alpha^*}}{\alpha^*}$. Instance $\mathcal{I}$ consists of only 2 type of orders each duplicated C times, i.e., $z_1 := (t_1 = 0, h_1 = 1)$ and $z_2 := (t_2 = 1^-, h_2 = e^{-\alpha^*})$ where we have C duplicate of each of them. Under this instance, the policy will dispatch all of the second type orders immediately and just keep holding all first types till their deadline, $t = 1$. However, the optimal offline policy will dispatch all the first type orders when the second type ones arrive and start holding them until the end of their deadline which is $t = 2^-$. As a result we will have $U_{\pi^{\mathsf{exp}(\alpha^*)}}(\mathcal{I}) = C$, while $\mathsf{OPT}(\mathcal{I}) = C + Ce^{-\alpha^*} = C(1 + e^{-\alpha^*})$. Therefore, their ratio will be:

$$\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)}) \leq \frac{U_{\pi^{\mathsf{exp}(\alpha^*)}}(\mathcal{I})}{\mathsf{OPT}(\mathcal{I})} = \frac{1}{1 + e^{-\alpha^*}} = \frac{1 - e^{-\alpha^*}}{\alpha^*}.$$

Note that the last equality comes from the choice of α^*, i.e., $1 - \alpha^* = e^{-2\alpha^*} \to \alpha^* = 1 - e^{-2\alpha^*} = (1 - e^{-\alpha^*})(1 + e^{-\alpha^*})$. This concludes the tightness of the competitive ratio of this policy, i.e., $\mathcal{CR}(\pi^{\mathsf{exp}(\alpha^*)}) = \frac{1-e^{-\alpha^*}}{\alpha^*}$.

□

Appendix D: Proofs in Section 5

D.1. Proof of Proposition 3

Proof of Proposition 3. Consider the continuum of arrival of orders over the time interval $[0,1]$, where at each time t we input C duplicates of order $z_t := (t, h_t := 1-t), \forall t \in [0,1]$. It is not hard to see that an optimal policy is to hold all the C duplicates of the order z_0 until the end of their lifespan, i.e., $t=1$. Therefore, we get $\mathsf{OPT} = C$. On the other hand, $\mathsf{Skyline}$ not only will keep holding all C duplicates of the z_0 until $t=1$, but also it will hold all the dispatched orders for an infinitesimal duration of time at their deadline. As a result, we would have $\mathsf{Skyline} = C(1+\int_1^2 h_{t-1}dt) = C\frac{3}{2}$. This will tell us that there is at least a gap of $\frac{2}{3}$ on the ratio of OPT over $\mathsf{Skyline}$ under worst-case instance, so as on the ratio of any policy π against the $\mathsf{Skyline}$. □

D.2. Proof of Proposition 4

Proof of Proposition 4. We prove this proposition by providing a strategy for the adversary that any feasible deterministic online algorithm will perform poorly under that. In particular, given any feasible deterministic online algorithm π, adversarial instance $\mathcal{I}^{Adv}$ is constructed in the following manner. Suppose the first order arrives with $z_1 := (t_1 = 0, h_1 = 1)$. After that, the second order will arrive with parameters $z_2 := (t = x^*, h = x^*)$, where $x^* \approx 0.597$ is the unique solution to $\frac{1}{2x} = \frac{x(2-x)}{1}$ on $[0,1]$. At time x^* any algorithm needs to decide which one of the order 1 or 2 should be dispatched from the system. If π decided to dispatch order 2 and continue on holding order 1, the adversary will cut any further arrival to the system, i.e., make order 2 the last arrived order. On the other hand, if the policy decided to dispatch order 1 and start holding order 2, then at time $t=1$, the adversary will restart the instance and repeat this process for a new unit time, i.e., it assumes $t=0$, and it will input to the system the two orders 1 and 2 as specified earlier.

Now we claim that under instance $\mathcal{I}^{Adv}$ - no matter how many rounds are passed based on the response of the algorithm, where in each round k the adversary inputs a pair of orders (order $z_{2k-1} := (2(k-1), 1)$ and order $z_{2k} := (2(k-1)+x^*, x^*)$) - the ratio of the total reward that π has accumulated compared to that of the optimal offline would be equal to $x^*(1-x^*)$.

To prove this claim, suppose a total of K rounds have passed until the instance $\mathcal{I}^{Adv}$ got terminated by the adversary. This means that the algorithm has accepted to hold all new arrivals until the arrival of the next order, except for the last order $2K$ for which the algorithm has chose to dispatch it immediately and continued holding the order $2K-1$ until its deadline $t_{2K-1}+1 = K$.

As for the optimal offline, it will only hold the odd orders and dispatches all even orders immediately, except the last order $2K$. More specifically, all orders $2i-1$ for all $i \in [K-1]$ will be held in the system for a whole unit of time, order $2K-1$ will be held for period with length x^*, and lastly, order $2K$ will be held for a whole unit of time again.

Accordingly, the utility that algorithm accumulates in each round $i \in [K-1]$ (the time interval $[i-1, i)$) is equal to $x^*(2-x^*)$, while it accumulated a total reward of 1 from the order $2K-1$, and nothing more. On the other hand, the optimal offline is accumulating a total reward of 1 in each of the rounds $i \in [K-1]$, as well as a total of x^* from each of the orders $2K-1$ and $2K$.

Finally, we get that the ratio of the reward of the algorithm to optimal offline in each of the rounds $i \in [K-1]$ is $\frac{x^*(2-x^*)}{1}$, while the ratio of their total reward after time $t = K-1$ would be $\frac{1}{2x^*}$. Now due to the fact that x^* satisfies $\frac{1}{2x^*} = \frac{x^*(2-x^*)}{1}$, we can conclude that the ratio of total reward of the algorithm compared to that of the optimal offline under such instance $\mathcal{I}^{Adv}$ is $x^*(2-x^*)$, concluding the proof of the claim. Consequently, our proposition is proved from the fact that the competitive ratio is, in fact, the minimum ratio under such adversarial instances (strategies). □

D.3. Proof of Proposition 5

Proof of Proposition 5. We will provide a mixed strategy for the oblivious adversary. Recall the strategy the adversary was using in the proof of Proposition 4 in Subsection D.2. It was a repeated round instance, where in each round k, the adversary was inputting the two orders $z_1 := (t_1 = k-1, h_1 = 1)$ and $z_2 := (t = k-1+x^*, h = x^*)$, where $x^* \approx 0.597$ was the solution to $\frac{1}{2x} = \frac{x(2-x)}{1}$. In there, the stopping time K (end of the last round K when the adversary terminates sending any new round after that) was computed based on the decisions of the policy π, which was deterministic.

Now that the policy π can be a randomized policy, we consider the instance $\mathcal{I}$ in which the adversary will input the same pair of orders in each round k - with parameters exactly as described in the deterministic version in Subsection D.2 - with probability $\frac{1}{2}$ independently at random, and with probability $\frac{1}{2}$, the adversary skips feeding the pair in round k and awaits until the time of next round $k+1$ to arrive and repeats this coin flipping process forever. Under such a random instance, it is not hard to see that at any time k (the end of round k), if the next round would be skipped then the optimal algorithm will continue holding the order it had from round k until its deadline, and if the round $k+1$ arrives, then it is optimal to switch to the fresh z_1 (type-1) order of the new round.

With this in mind, we can see that at any time $k-1+x^*$ (the time in round k that the algorithm should decide whether or not to switch from holding the type-1 order z_1 to the type-2 order z_2) no matter what decision it takes, the competitive ratio of its decision (corresponding to the total reward achieved only from the orders in round k) will be the same and equal to $\frac{1+x^*(2-x^*)}{2} < \frac{1+0.8376}{2} \approx 0.92$, and for the next non-skipped rounds everything will be reset due to the fact that we showed any optimal policy will switch at the beginning of a non-skipped round.

Accordingly, with this mixed strategy, the adversary has made any feasible online policy π to be indifferent in the middle-round decisions. Thus, showing us that the competitive ratio of any feasible randomized online policy would be now upper bounded by $\frac{1+x^*(2-x^*)}{2} < \frac{1+0.8376}{2} \approx 0.92$. □

Appendix E: Proofs of Section 6

E.1. Proofs in Subsection 6.1

E.1.1. Proof of Theorem 3

Proof of Theorem 3. We prove that the following chain of inequalities hold:

$$U_{\pi^{\mathsf{red}}}(\mathcal{I}) \geq U_{\widetilde{\pi}}(\widetilde{\mathcal{I}}) \geq \widetilde{\mathcal{CR}} \times \mathsf{OPT}(\widetilde{\mathcal{I}}) \geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L}} \times \mathsf{OPT}(\mathcal{I}).$$

To do so, let us prove each of the three inequalities one by one:

1. $U_{\pi^{\mathsf{red}}}(\mathcal{I}) \geq U_{\widetilde{\pi}}(\widetilde{\mathcal{I}})$: We prove that for any order i, the total reward corresponding to it that π^{red} acquires is higher than the amount that $\widetilde{\pi}$ acquires. To see this, we consider all possible cases in the following:

(a) $\ell_i \geq \sqrt{L}$: In this case, we know $d_i^{\pi^{\mathsf{red}}} = d_i^{\widetilde{\pi}}$, and the arrival time and the reward rate for such an order were the same under both $\widetilde{\mathcal{I}}$ and $\mathcal{I}$, hence they both receive exactly the same reward from order i.

(b) $\ell_i < \sqrt{L},\ d_i^{\widetilde{\pi}} \leq t_i + \ell_i$: In this case, we again know $d_i^{\pi^{\mathsf{red}}} = d_i^{\widetilde{\pi}}$. Additionally, notice that we had $h_i > \widetilde{h}_i$; therefore, we can again get $h_i(d_i^{\pi^{\mathsf{red}}} - t_i) \geq \widetilde{h}_i(d_i^{\widetilde{\pi}} - t_i)$.

(c) $\ell_i < \sqrt{L},\ d_i^{\widetilde{\pi}} > t_i + \ell_i$: In this case, we have $d_i^{\pi^{\mathsf{red}}} = t_i + \ell_i$. As a result, we get $h_i(d_i^{\pi^{\mathsf{red}}} - t_i) = h_i\ell_i = \widetilde{h}_i\widetilde{\ell}_i \geq \widetilde{h}_i(d_i^{\widetilde{\pi}} - t_i)$, which is again what we wanted to prove.

2. $U_{\widetilde{\pi}}(\widetilde{\mathcal{I}}) \geq \widetilde{\mathcal{CR}} \times \mathsf{OPT}(\widetilde{\mathcal{I}})$: Proof of this part is just by definition, i.e., $\widetilde{\mathcal{CR}}$ was, in fact, the competitive ratio of $\widetilde{\pi}$ under the homogeneous lifespan setting, which subsumes $\widetilde{\mathcal{I}}$.

3. $\mathsf{OPT}(\widetilde{\mathcal{I}}) \geq \frac{1}{\sqrt{L}} \times \mathsf{OPT}(\mathcal{I})$: In order to prove this last inequality, we will provide a feasible policy π under the homogeneous instance $\widetilde{\mathcal{I}}$ whose utility satisfies: $\mathsf{OPT}(\widetilde{\mathcal{I}}) \geq U_\pi(\widetilde{\mathcal{I}}) \geq \frac{1}{\sqrt{L}} \times \mathsf{OPT}(\mathcal{I})$. Note that the first inequality automatically follows by definition of optimality of $\mathsf{OPT}(\widetilde{\mathcal{I}})$ and feasibility of π under $\widetilde{\mathcal{I}}$. To define behavior of π under $\widetilde{\mathcal{I}}$ uniquely, again we only need to determine its dispatch times for all order in $\widetilde{\mathcal{I}}$. Thus, we set $d_i^\pi = \min(d_i^{\mathsf{OPT}(\mathcal{I})}, t_i + \widetilde{\ell}_i)$. By this way of defining the dispatch time as well as the fact that $\mathsf{OPT}(\mathcal{I})$ was feasible under $\mathcal{I}$ (w.r.t. capacity and lifespan constraints), we can easily see that π is feasible under $\widetilde{\mathcal{I}}$. The last remaining part to show is the inequality $U_\pi(\widetilde{\mathcal{I}}) \geq \frac{1}{\sqrt{L}} \times \mathsf{OPT}(\mathcal{I})$. Similarly, we will show that for any order i, the total amount of reward corresponding to i under both $\pi(\widetilde{\mathcal{I}})$ and $\mathsf{OPT}(\mathcal{I})$, satisfies the inequality. Consider the following two possible cases:

(a) $\ell_i \geq \sqrt{L}$: For this class of orders, we have $\widetilde{h}_i = h_i$ and $d_i^\pi - t_i = \min(d_i^{\mathsf{OPT}(\mathcal{I})} - t_i, \sqrt{L}) \geq \frac{1}{\sqrt{L}}(d_i^{\mathsf{OPT}(\mathcal{I})} - t_i)$, because $d_i^{\mathsf{OPT}(\mathcal{I})} - t_i \leq L$. As a consequence, we get $\widetilde{h}_i(d_i^\pi - t_i) \geq h_i \frac{1}{\sqrt{L}}(d_i^{\mathsf{OPT}(\mathcal{I})} - t_i)$, which is exactly what we wanted to show.

(b) $\ell_i < \sqrt{L}$: For this other class of orders, we have $d_i^\pi = d_i^{\mathsf{OPT}(\mathcal{I})}$ and $\widetilde{h}_i = \frac{h_i\ell_i}{\sqrt{L}} \geq \frac{h_i}{\sqrt{L}}$, because $\ell_i \geq 1$. As a result, we get $\widetilde{h}_i(d_i^\pi - t_i) \geq \frac{h_i}{\sqrt{L}}(d_i^{\mathsf{OPT}(\mathcal{I})} - t_i)$, which is again what we wanted to show.

Putting all these three pieces of chain inequalities together, we conclude the proof of the theorem.

□

E.1.2. Proof of Proposition 6

Proof of Proposition 6. It is enough to provide an adversary instance generation strategy under which no deterministic feasible online algorithm is able to beat $\frac{1}{\sqrt{L}}$ fraction of the OPT utility. The bad instance $\mathcal{I}$ will be generated as follows. Fix a deterministic feasible online algorithm π. At time $t = 0$, the adversary will simultaneously input two orders $z_1 = (t_1 = 0, h_1 = \sqrt{L}, \ell_1 = 1)$ and $z_2 = (t_2 = 0, h_2 = 1, \ell_2 = L)$. If π chooses to hold z_1, then the adversary will terminate the instance, i.e., will send no more arrivals to the system. Otherwise, i.e., if π chooses to hold z_2, then at time $t = 1$, the adversary imagines $t = 0$ and repeat the entire pair arrival again, with two new orders z_3 and z_4 that are defined in exactly the same way as above, except for their arrival time which is now 1 for both of them. The adversary will continue on inputting these pairs of orders (at integer times) until the first time, say at round k, that the algorithm decides to hold the order z_{2k-1} and dispatch order z_{2k}, after which the adversary will send no more arrivals to the system.

To see that why the utility ratio of the algorithm against optimal offline under such an instance is at most $\frac{1}{\sqrt{L}}$, suppose the instance consists of a total of K repeated rounds before it gets terminated. The algorithm must have chosen to hold all even orders $z_{2k}, \forall k \in [K-1]$ as well as order z_{2K-1}, while the optimal offline - under such instance - will choose to hold all odd orders $z_{2k-1}, \forall k \in [K-1]$ together with the last order z_{2K}. Thus, the utility of the algorithm $U_\pi(\mathcal{I}) = (K-1) + \sqrt{L}$, whereas that of the optimal offline would be $\mathsf{OPT}(\mathcal{I}) = (K-1)\sqrt{L} + L$, which generates a competitive ratio of at most $\mathcal{CR}(\pi) \leq \frac{1}{\sqrt{L}}$ and concludes our proof. □

E.2. Proofs in Subsection 6.2

E.2.1. Proof of Lemma 6

Proof of Lemma 6. The proof of the lemma concludes from the following chain of inequalities:

$$\begin{aligned} U_{\pi_K^{\mathsf{div}}}(\mathcal{I}; C) &= \sum_{k\in[K]} U_{\pi^k}\left(\mathcal{I}_k; \lfloor \frac{C}{K} \rfloor\right) \\ &\geq \sum_{k\in[K]} \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \mathsf{OPT}\left(\mathcal{I}_k; \lfloor \frac{C}{K} \rfloor\right) \\ &\geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \sum_{k\in[K]} \frac{\lfloor \frac{C}{K} \rfloor}{C} \mathsf{OPT}(\mathcal{I}_k; C) \\ &\geq \frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}} \frac{\lfloor \frac{C}{K} \rfloor}{C} \mathsf{OPT}(\mathcal{I}; C). \end{aligned}$$

Before delving into the rationale behind each inequality, it is essential to recall that the second argument (C or $\lfloor \frac{C}{K} \rfloor$) in the utility functions ($U_\pi(\mathcal{I}; C)$ or $\mathsf{OPT}(\mathcal{I}; C)$) represents the system capacity under which the algorithm operates. For the sake of brevity, we had omitted this parameter so far, as our analysis focused solely on a single value of capacity. However, with the introduction of distinct capacities for our sub-problems compared to the original problem, it becomes necessary to reintroduce this parameter for clarity. Now let us state the reasoning behind each step of this chain of inequalities:

1. The first equality holds because π_K^{div} is exactly emulating all of the π^k, $\forall k \in [K]$, simultaneously.

2. The second inequality holds from the fact that the competitive ratio of each π^k (with capacity $\lfloor \frac{C}{K} \rfloor$) under $\mathcal{I}_k$ was $\frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}}$.

3. The third inequality holds from the simple observation that if you multiply the capacity of OPT by some factor, its utility will be increased by at most that factor.

4. The fourth inequality results from the following two facts. First, $\mathcal{I} = \cup_{k\in[K]} \mathcal{I}_k$. Second, for any $k \in [K]$, the way $\mathsf{OPT}(\mathcal{I}; C)$ holds just the orders in $\mathcal{I}_k$, would itself be a feasible policy under the instance $\mathcal{I}_k$ when capacity is still C. Therefore, the share of the utility $\mathsf{OPT}(\mathcal{I}; C)$ that comes from the orders in $\mathcal{I}_k$ is less than the $\mathsf{OPT}(\mathcal{I}_k; C)$. Finally, by taking the summation over all categories, we get the inequality we want, since from the first fact we are sure that we have accounted for each order's share in $\mathsf{OPT}(\mathcal{I}_k; C)$ at least once.

□

E.2.2. Proof of Theorem 4

Proof of Theorem 4. From Lemma 6, we know that

$$\mathcal{CR}\left(\pi^{\mathsf{div}}_{K^*}\right) \geq \max_{K\in[C]}\left\{\frac{\widetilde{\mathcal{CR}}}{\sqrt{L^{\frac{1}{K}}}}\frac{\lfloor\frac{C}{K}\rfloor}{C}\right\}.$$

Hence, it is sufficient to show that whenever $C=\Omega(\log(L))$, we have

$$\max_{K\in[C]}\left\{\frac{1}{\sqrt{L^{\frac{1}{K}}}}\frac{\lfloor\frac{C}{K}\rfloor}{C}\right\}=\Omega\left(\frac{1}{\log(L)}\right).$$

Let w be a small constant such that $C\geq w\log(L)$. Now let $v\ll w$ also be a constant, which is much smaller than w, say $\frac{w}{v}>11$. Now let $\hat{K}:=\lceil v\log(L)\rceil<\frac{w\log(L)}{10}\leq\frac{C}{10}$, where L is sufficiently large ($v\log(L)>10$). As a consequence, we will get

$$\frac{1}{\sqrt{L^{\frac{1}{\hat{K}}}}}\geq\frac{1}{e^{\frac{1}{2v}}}=\Omega(1),\quad \frac{\lfloor\frac{C}{\hat{K}}\rfloor}{C}\geq\frac{\frac{C}{\hat{K}}-1}{C}=\left(\frac{1}{\hat{K}}-\frac{1}{C}\right)=\frac{0.9}{\hat{K}}+\left(\frac{1}{10\hat{K}}-\frac{1}{C}\right)\geq\frac{0.9}{\hat{K}}=\Omega\left(\frac{1}{\log(L)}\right).$$

Thus, we get $\max_{K\in[C]}\left\{\frac{1}{\sqrt{L^{\frac{1}{K}}}}\frac{\lfloor\frac{C}{K}\rfloor}{C}\right\}\geq\frac{1}{\sqrt{L^{\frac{1}{\hat{K}}}}}\frac{\lfloor\frac{C}{\hat{K}}\rfloor}{C}=\Omega\left(\frac{1}{\log(L)}\right)$, completing the proof of the theorem. □

Appendix F: Proofs in Section 7

F.1. Proof of Lemma 7

Proof of Lemma 7. The two equalities are simply from assumption (11) and how we define r_z, which equals $\mathfrak{h}(t)e^{\alpha t}$. The two inequalities can be derived by setting $t'\uparrow t+1$ and $t'\downarrow t$, respectively. Lastly, if the function $\frac{e^{-\alpha t''}}{\mathfrak{r}(t'')}$ is non-increasing in t'' on $[0,1]$, then we claim that

$$\frac{\int_0^{t'-t}e^{-\alpha t''}\,dt''}{\int_0^{t'-t}\mathfrak{r}(t'')\,dt''} \tag{EC.4}$$

is also non-increasing in t' on $[t,t+1]$. This is due to the fact that if a ratio of two functions $\frac{a(t)}{b(t)}$ is non-increasing in t and $b(t)>0$ for all $t\geq 0$, then $\frac{\int_0^T a(t)dt}{\int_0^T b(t)dt}$ would also be non-increasing in T for all $T\geq 0$. To see this, note that

$$\frac{d\left(\frac{\int_0^T a(t)dt}{\int_0^T b(t)dt}\right)}{dT}=\frac{a(T)\int_0^T b(t)dt-b(T)\int_0^T a(t)dt}{\left(\int_0^T b(t)dt\right)^2}=\frac{b(T)}{\int_0^T b(t)dt}\left[\frac{a(T)}{b(T)}-\frac{\int_0^T a(t)dt}{\int_0^T b(t)dt}\right],$$

where

$$\frac{\int_0^T a(t)dt}{\int_0^T b(t)dt}=\frac{\int_0^T b(t)\frac{a(t)}{b(t)}dt}{\int_0^T b(t)dt}\geq\frac{\int_0^T b(t)\frac{a(T)}{b(T)}dt}{\int_0^T b(t)dt}=\frac{a(T)}{b(T)},$$

which implies $\frac{d\left(\frac{\int_0^T a(t)dt}{\int_0^T b(t)dt}\right)}{dT}\leq 0$. The monotonicity of (EC.4) implies that the minimum value for each of γ_o and γ_a's min operator is, in fact, achieved when $t'\uparrow t+1$ and $t'\downarrow t$, respectively, thereby making the inequalities hold with equality and completing the proof of the lemma. □